

General Sir John Kotelawala Defense University

Faculty of Computing

Artificial Intelligence

Group Assignment

Final Report

Project Title: AI-Powered University Classroom Utilization Optimizer

Names and Degree Programmes of Group Members :

No.	Index No	Degree Programme	Name
1	D/BIT/24/0044	Information Technology	HAID Harshani
2	D/COE/25/0023	Computer Engineering	HDMC Nawarathna
3	D/BIT/24/0069	Information Technology	HMHP Herath
4	D/BCS/25/0015	Computer Science	LSR Gunathunga

Table of Content

1. Abstract **2**
2. Introduction **2**
3. Problem Background **3**
4. Problem Statement **4**
5. Objectives **5**
5.1 Main Objective 5
5.2 Specific Objectives 5
6. Selected AI Techniques **6**
7. Justification of AI Techniques **7**
7.1 Random Forest Regressor 7
7.2 Genetic Algorithm 7
7.3 MAUT and Content-Based Filtering 7
7.4 Rule-Based MCDM and Dynamic Heuristics 7
7.5 Explainable AI 7
8. System Design **8**
8.1 Overall System Architecture **8**
8.2 Main System Workflow 9
9. Implementation Details **10**
9.1 Attendance Prediction Implementation **10**
9.2 Genetic Algorithm Implementation 11
The Genetic Algorithm represents a possible allocation using: 11
9.3 Study-Space Recommendation 12
9.4 Emergency Classroom Reallocation 12
10. Testing and Results **12**
10.1 Attendance Prediction Results 12
10.2 Classroom Optimization Testing 13
10.3 Functional Testing 13
10.4 Performance Testing 13
11. Limitations and Discussions **13**
12. Ethical and Social Considerations **14**
13. Conclusion **14**
14. References **15**
15. Individual Contribution Report **16**

1. Abstract

General Sir John Kotelawala Defence University Classroom allocation is often performed on a fixed number of students and fixed timetables,which does not correctly represent the actual number of students at a specific classroom at a given time. So these problems can lead to wasted class spaces, unused space, overcrowding and challenges when there are unexpected class changes.

The goal of this project is to build an AI-Powered University Classroom Utilization Optimizer designed for the Faculty of Computing at General Sir John Kotelawala Defence University. The system integrates machine learning, artificial intelligence and optimization techniques to optimize the use of and allocation of classrooms. Student attendance is predicted using a Random Forest Regressor, which is a supervised machine learning algorithm. Random Forest Regressor is a machine learning supervised algorithm used to predict attendance of students based on the session related factors.Genetic Algorithm is then employed to determine suitable classroom allocation,taking into account the capacity of the classrooms, constraints on the timetable, utilization of the classrooms and travelling distance. The system offers study space suggestions and helps in emergency reallocation of classrooms.

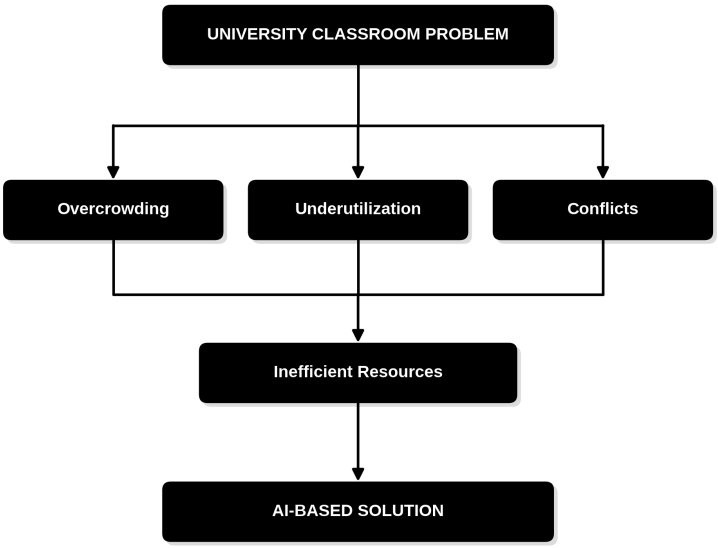
The proposed system has three layers, React frontend, Spring Boot backend, Al microservices (FastAPI) and MySQL database. Explainable Al techniques are integrated to give the reasons behind key predictions and recommendations that are understandable to humans. The model predicted the students' attendance with an R score of 0.9951 and an MAE of 1.45 students for the test set reported.

The system illustrates the integration of AI-based prediction and optimization to assist in more efficient and flexible classroom management in university.

2. Introduction

This project proposes AI-Powered University Classroom Utilization Optimizer design for the Faculty of Computing at General Sir John Kotelawala Defence University. Effective classroom management is an important requirement for the Faculty of Computing at General Sir John Kotelawala Defence University because university classrooms are limited resources that need to be allocated efficiently among different courses, students,special events and academic sessions. So KDU traditional classroom allocation approaches generally depend on fixed enrollment information and predefined schedules.Then the actual number of students attending a session can vary considerably depending on the course,time,weather conditions and the session related factors.

The proposed project addresses this issue by developing an Intelligent classroom utilization system for the Faculty of Computing at General Sir John Kotelawala Defence University.This system considers predicted attendance,classroom capacity,time table constraints,room availability and physical distance between buildings when making allocation decisions.



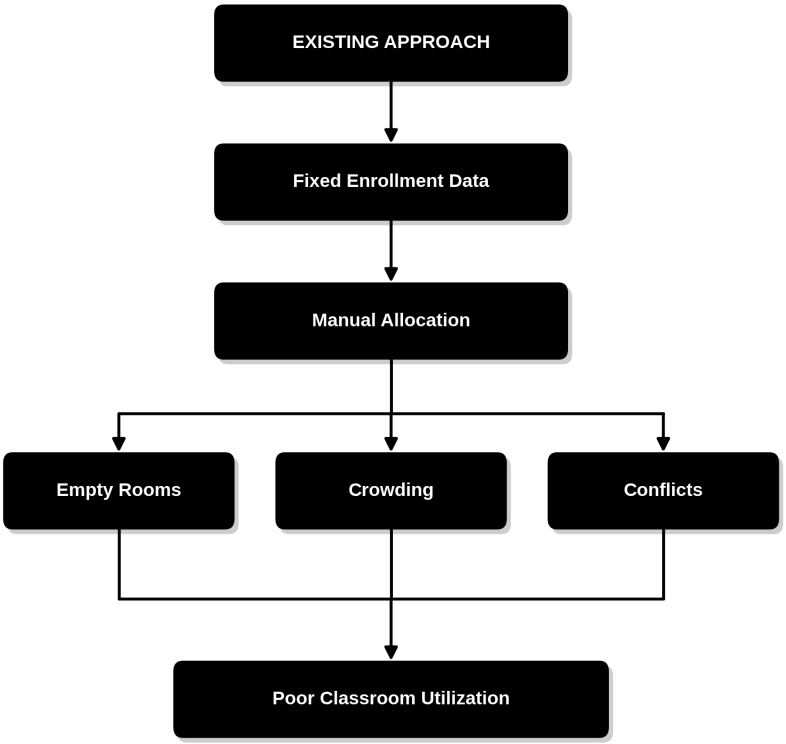
Our project combined several AI techniques rather than depending on a single algorithm.Random forest is used for attendance prediction while a genetic algorithm is used to optimise classroom allocations.Our system recommends additional recommendation and decision making techniques used to support study space reccomendations and emergency classroom reallocation.

3. Problem Background

The university has to manage a large number of classrooms, with a variety of courses, intakes, lecturers and time slots. If the class allocation is primarily based on the number of students enrolled in the class, the classroom used may not reflect the number of students that are likely to attend a specific class.

As an example, a course with a high enrolment of students, the actual attendance on a particular day may be low. Allocating a large classroom in such a situation can result in underutilization. Conversely, a small classroom could become overcrowded when a class has an unexpectedly large number of students.

The problem can be more complex when classrooms can be used at the same time by multiple intakes and multiple courses. There are additional challenges of managing available spaces such as classroom cancellations, timetable conflicts and unexpected changes.



Therefore the project aims to develop an intelligent system, which is capable of predicting attendance and then utilizing the prediction to allocate classrooms. Official timetables of intakes 41,42 and 43 along with the actual information from Google map coordinates and walking distance was added to make the system more representative of the actual University environment.

4. Problem Statement

The existing university classroom allocation process may rely on fixed enrollment numbers rather than the expected actual attendence of students.Some classrooms can remain utilised while other sessions may experience overcrowding.

The problem becomes more challenging when multiple courses and intakes require classrooms simultaneously when classrooms are cancelled or when timetable conflicts occur. So students may have difficulty identifying suitable vacant spaces for individual or group study.

There is a need for an intelligent classroom management solution that can predict expected attendance,optimize classroom allocation,recommend suitable study spaces and support emergency classroom allocation while considering important scheduling, capacity and distance constraints.

5. Objectives

5.1 Main Objective

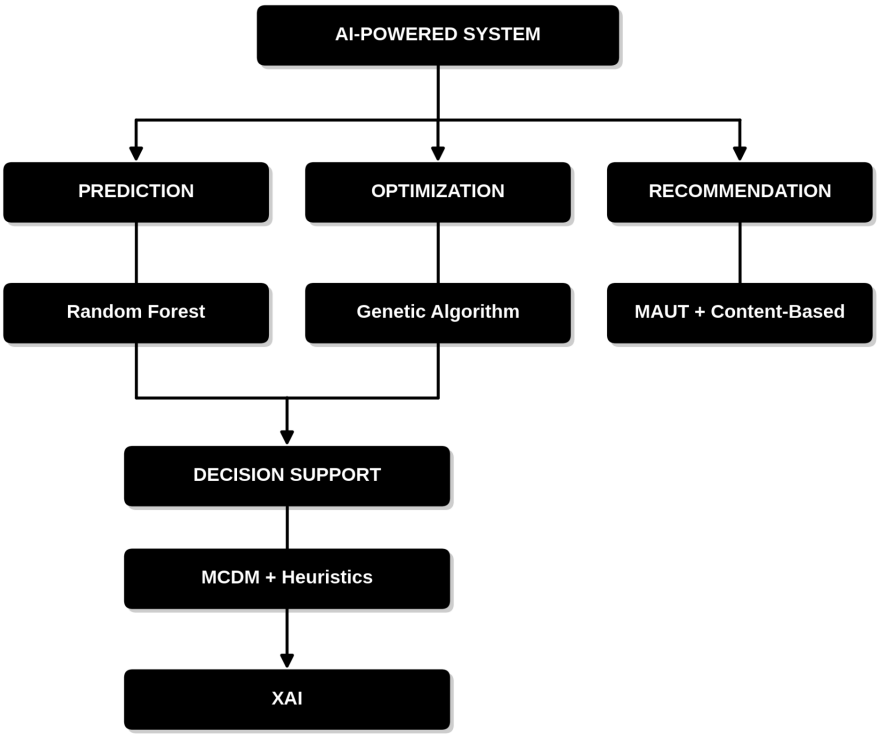
To build an AI-powered System that improves the usage of university classrooms by forecasting student presence and automatically improving the classroom allocation.

5.2 Specific Objectives

1. To create a machine learning model for predicting student attendance
2. To maximize efficient use of classroom allocation based on predicted attendance and classroom capacity.
3. To minimize the under utilization and overcrowding of classrooms.
4. To consider timetable and classroom constraints during allocation.
5. To reduce any unnecessary travel from relevant University buildings.
6. To recommend empty study spots available to students.
7. To establish an emergency reallocation of classroom facilities.
8. To explain in comprehensive way the important AI predictions and recommendations.
9. To analyze the performance of the proposed system by considering the performance measures.

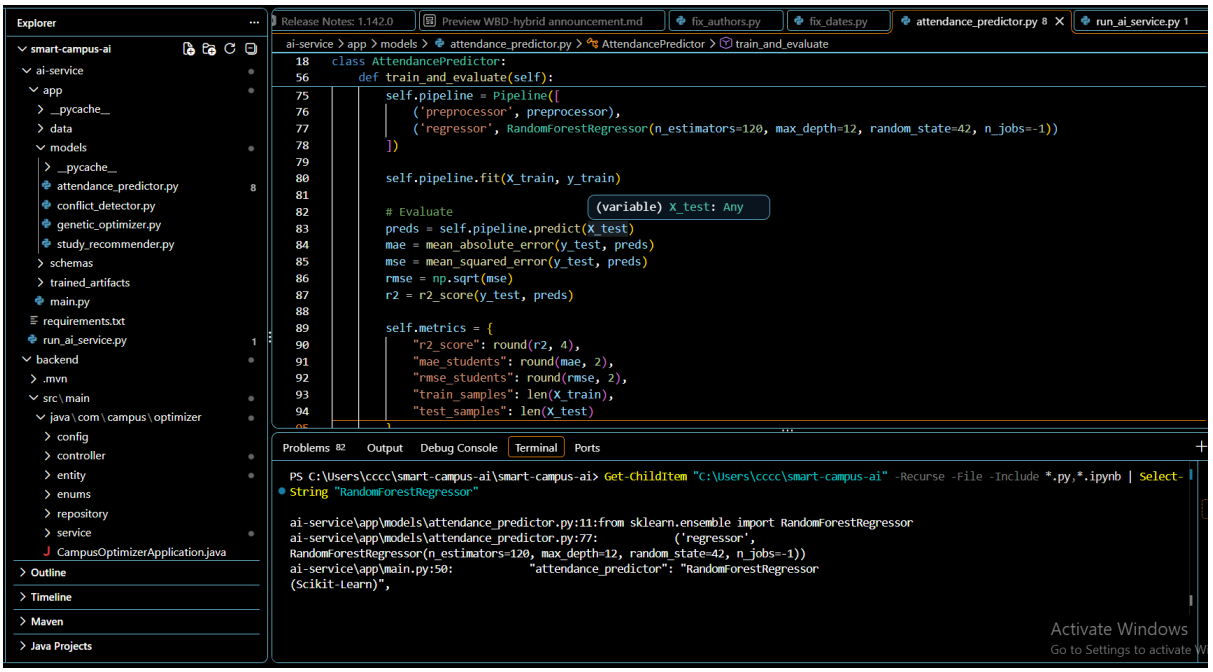
6. Selected AI Techniques

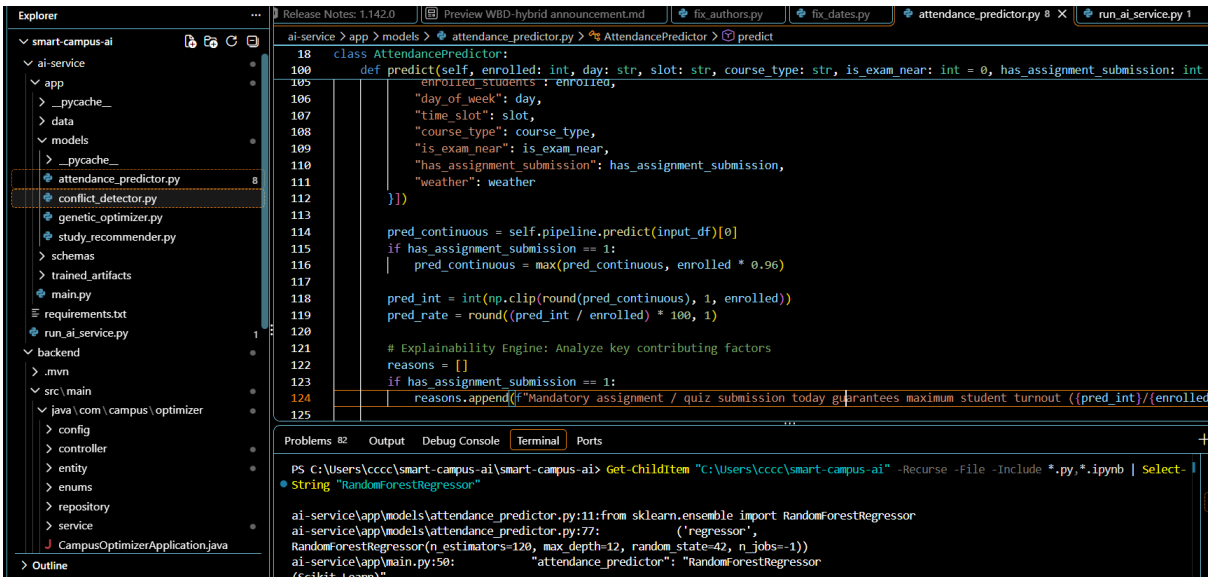
System Component	AI technique	Main Purpose
Attendance Prediction	Random Forest Regressor	Predict expected student attendance
Classroom Optimization	Multi Objective Genetic Algorithm	Generate improved classroom allocations
Study Space Recommendation	MAUT + Content Based Filtering	Recommend suitable vacant study spaces
Emergency Reallocation	Role-Based MCDM + Dynamic Heuristics	Handles unexpected classroom changes
Explanation Layer	Explainable AI	Explain predictions and recommendations



7. Justification of AI Techniques

7.1 Random Forest Regressor





This was selected for attendance prediction because student attendance can be affected by several factors and the relationships between these factors may not always be linear.

This model uses session related features such as course code ,number of enrolled students.day,time slot,course type,examination proximity, and weather conditions. Random Forest is suitable for this type of problem because it combines multiple decision trees and can capture complex relationships between input variables and attendance.

7.2 Genetic Algorithm

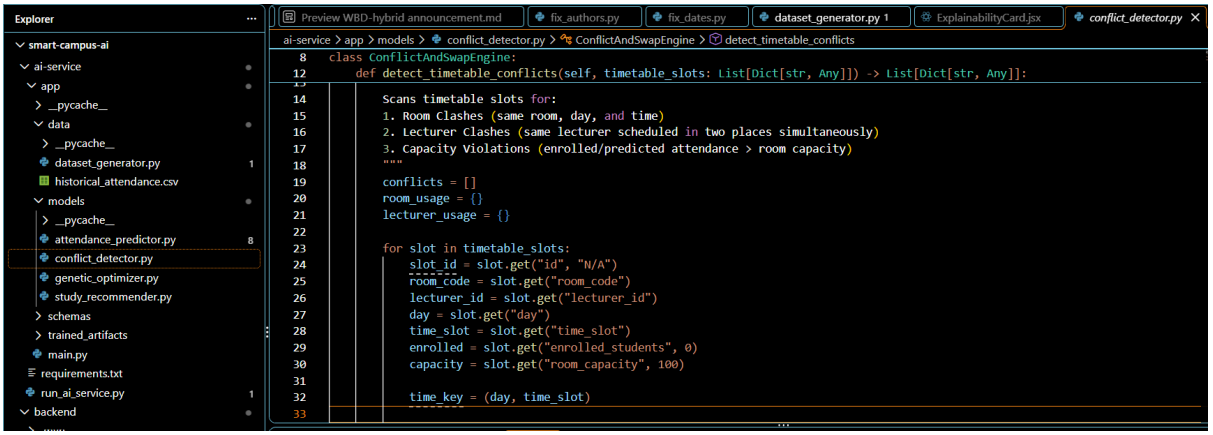


University classroom allocation is a complex optimization problem because there can be a large number of possible combinations of courses,classrooms, and time slots.

Searching every possible combination would be inefficient. A generic alogorithm used to search though possible solutions and gradually improve through selection,crossover and mutation.

The fitness function considers classroom utilization, constraint violations and unnecessary travelling distance.Allows the optimization process to balance several requirements rather than focusing on a single objective.

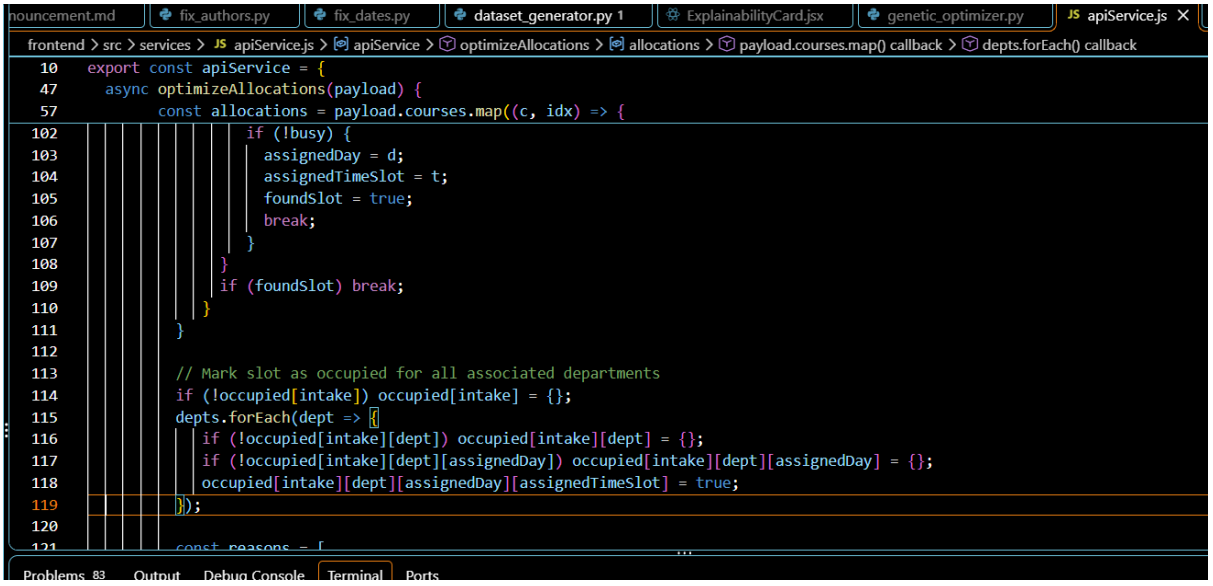
7.3 MAUT and Content-Based Filtering



The study space recommendation component uses Multi Attribute Utility Theory together with Content Based Filtering.

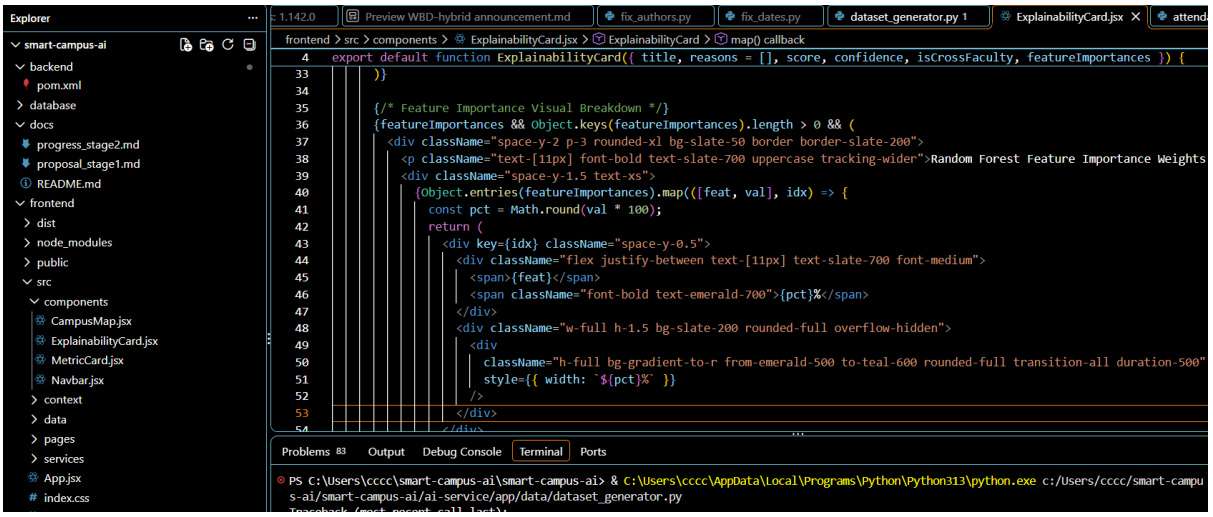
This approach allows the system to consider multiple characteristics of available study spaces and recommend locations that are more suitable for the user's requirements.

7.4 Rule-Based MCDM and Dynamic Heuristics



Unexpected events such as classroom cancellations require the system to identify alternative classrooms quickly.
Rule-Based Multi Criteria Decision Making and dynamic heuristics are used to evaluate available alternatives according to relevant constraints and generate an appropriate replacement.

7.5 Explainable AI

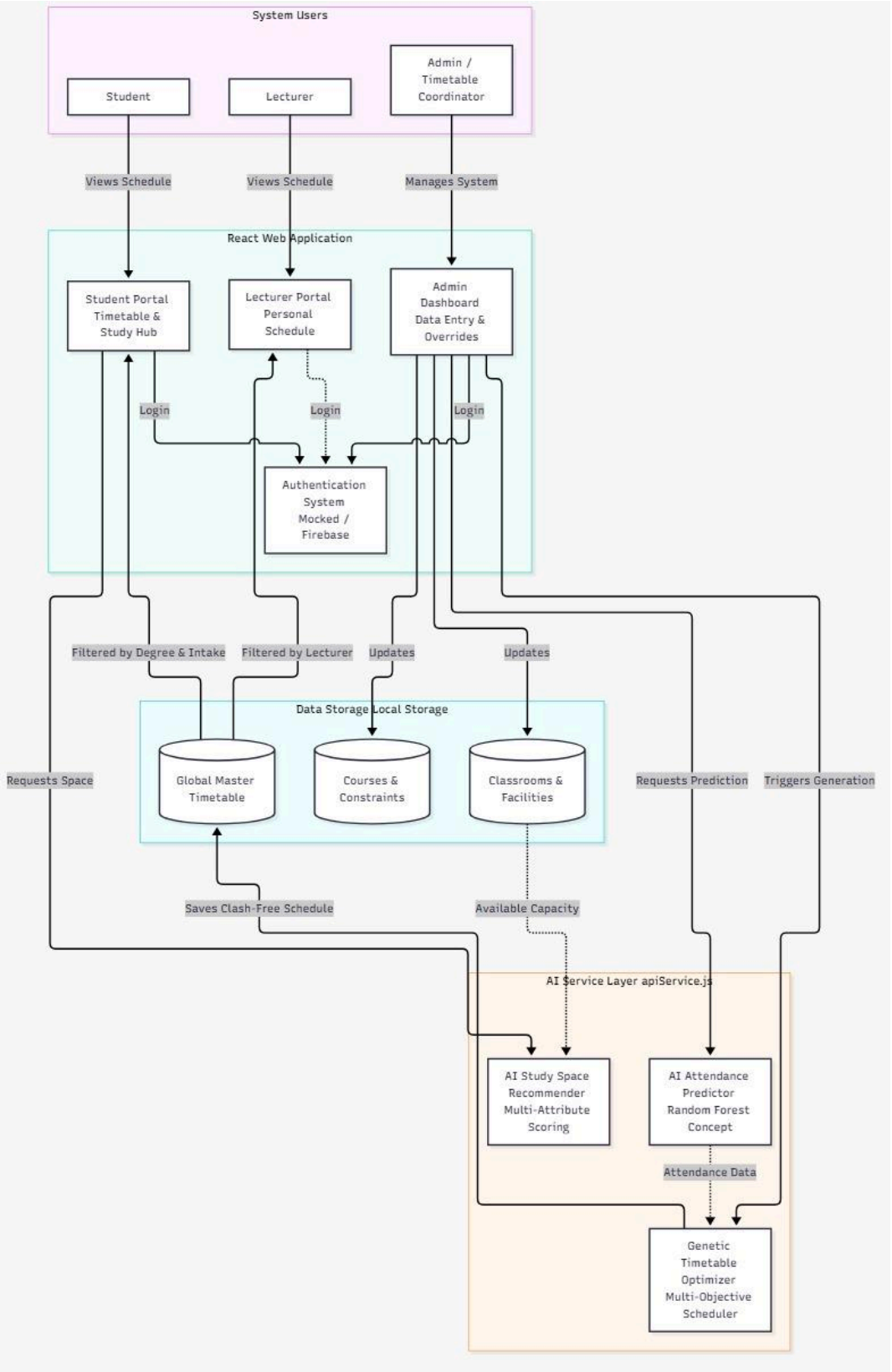


Explainable AI is included to make the system's predictions and recommendations easier to understand.

Instead of presenting only an output, the system provides information about the factors that influenced the prediction or recommendation. This improves transparency and helps users make informed decisions.

8. System Design

8.1 Overall System Architecture



The proposed system follows a three layer architecture :

Frontend
Frontend is created with React 18 and Tailwind CSS. It is the user interface to schedules, classroom allocation, recommendations and system results.

Backend
The Spring Boot backend handles the primary business logic and recommendations between the frontend, AI microservice, and database.

AI Microservice
The AI microservice is a FastAPI-based intelligent processing layer. It includes all the components of the Random Forest attendance prediction model, the Genetic Algorithm optimization engine, recommendation engine, MCDM rule engine, and XAI components.

Database
My SQL 8.0 used to store all data about courses, classrooms, timetables, attendance and other system data.

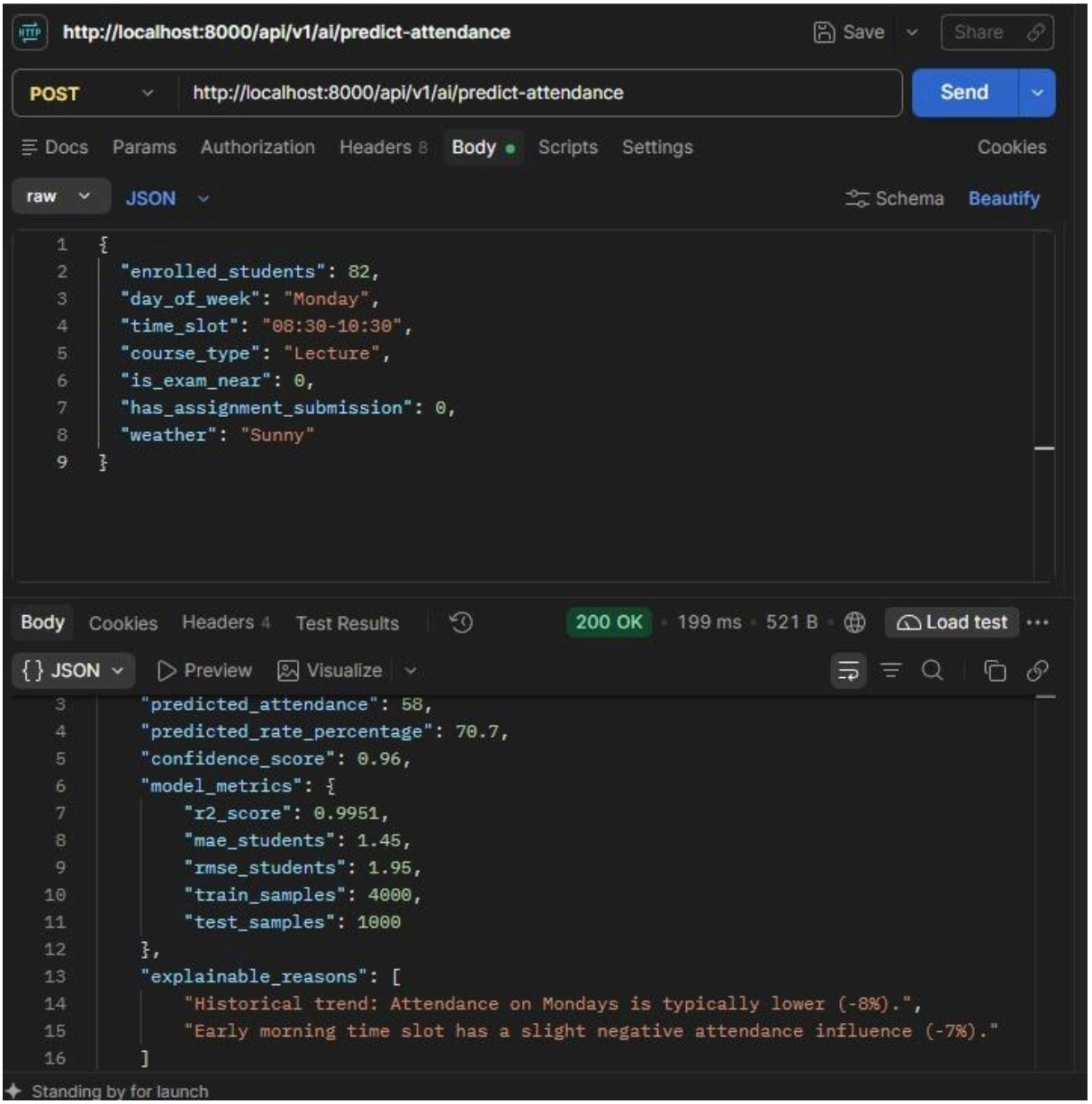
8.2 Main System Workflow

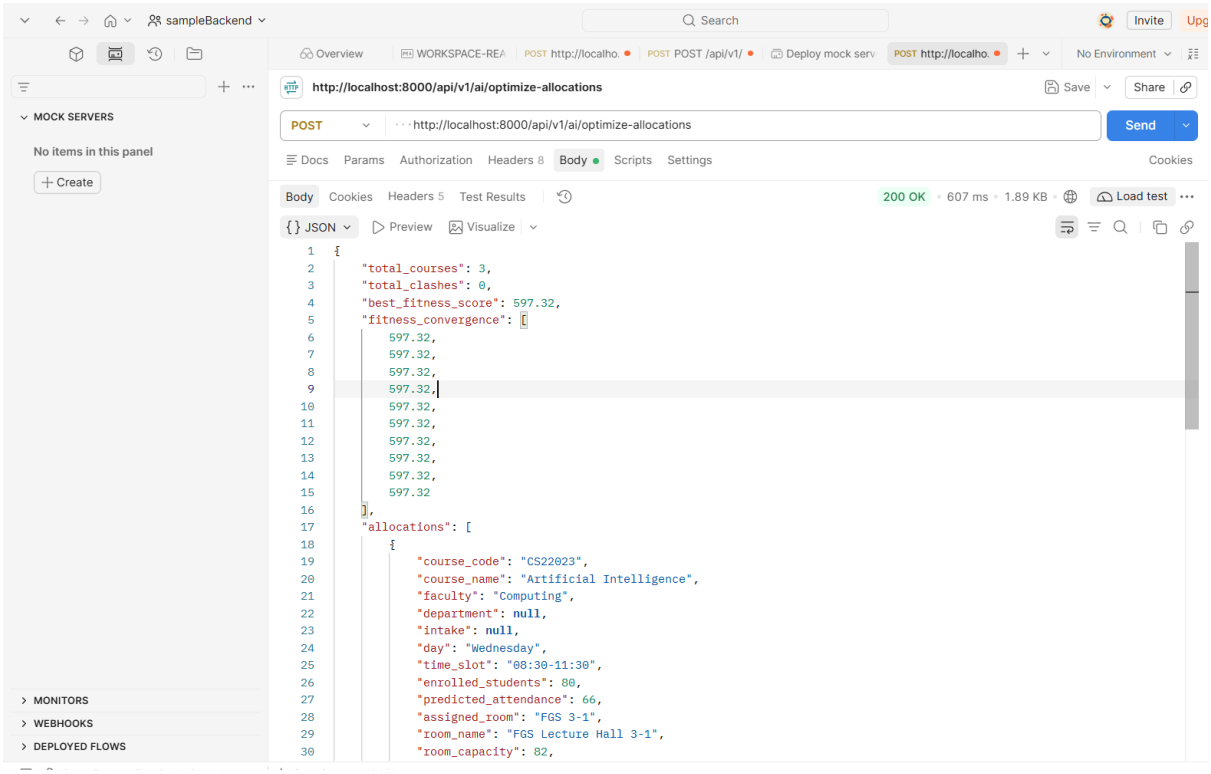
The overall process of the system may be described as follows:

- Relevant course, timetable, classroom, attendance data are passed to the system
- The attendance prediction model is used to predict attendance
- Predicted attendance is provided to the classroom allocation component.
- The optimization engine compute possible classroom allocations
- The system verifies capacity, timetable, availability and distance.
- The best suitable allocation to chosen
- The results are displayed through frontend
- The XAI component explains the results (Where applicable)

9. Implementation Details

9.1 Attendance Prediction Implementation

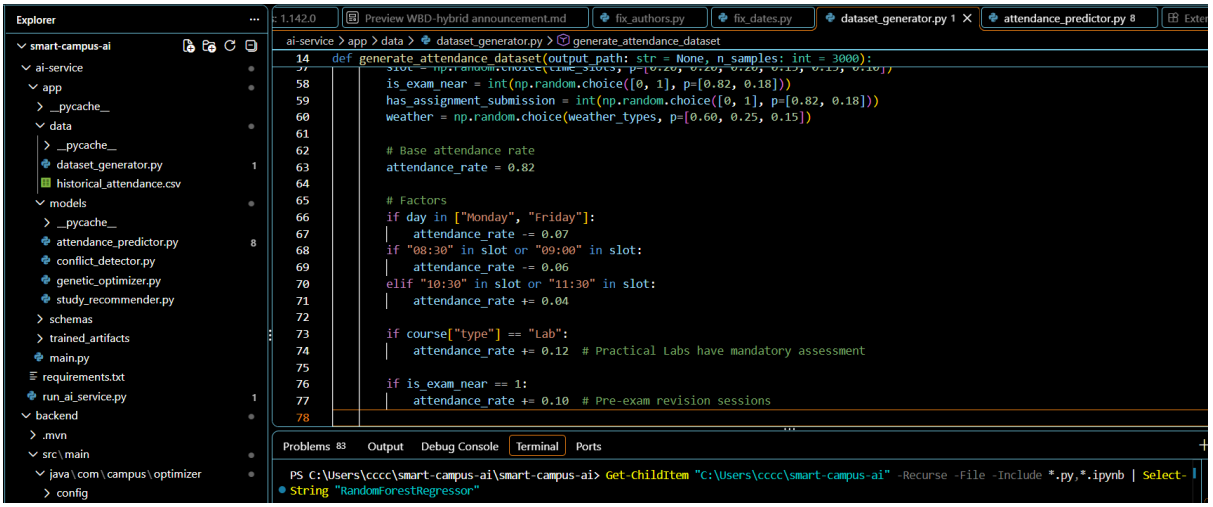




The attendance prediction component is implemented using a Random Forest Regressor. The reported training pipeline include:

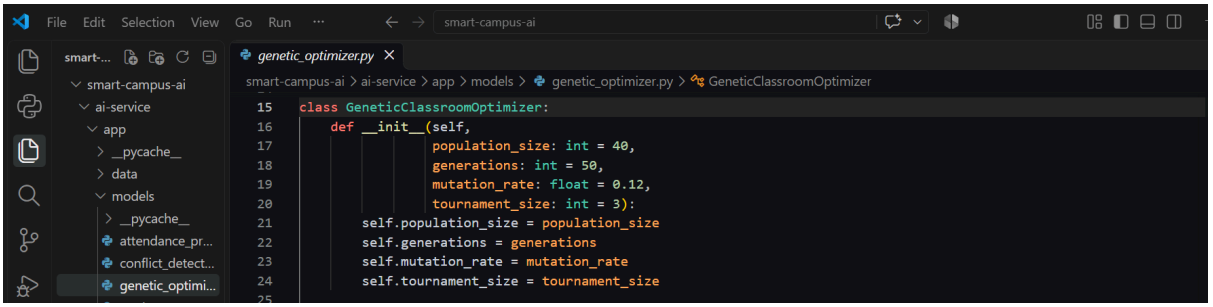
- Session-based dataset
- Seven input features
- One target variable- actual attendance
- Categorical feature encoding using OneHotEncoder
- Training and testing data split
- Random Forest model with 120 estimators
- Model serialization using Joblib
- Fast API based model inference

The attendance prediction dataset is prepared by selecting the relevant session-based input features and the actual attendance target variable. The data preprocessing stage includes the feature preparation, categorical encoding, and training testing split used by the model, as implemented in the project.

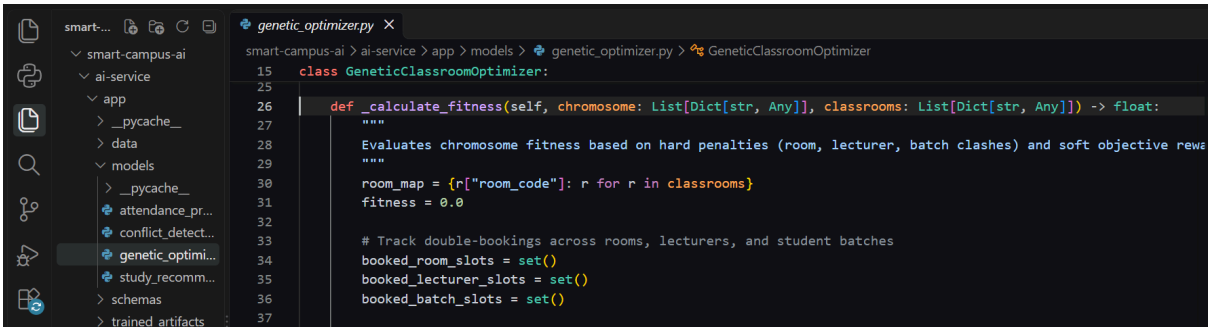


The trained model is integrated into the AI microservice so that predictions can be requested by the main system.

9.2 Genetic Algorithm Implementation







The Genetic Algorithm represents a possible allocation using:

(course_id, classroom_id, time_slot_id)

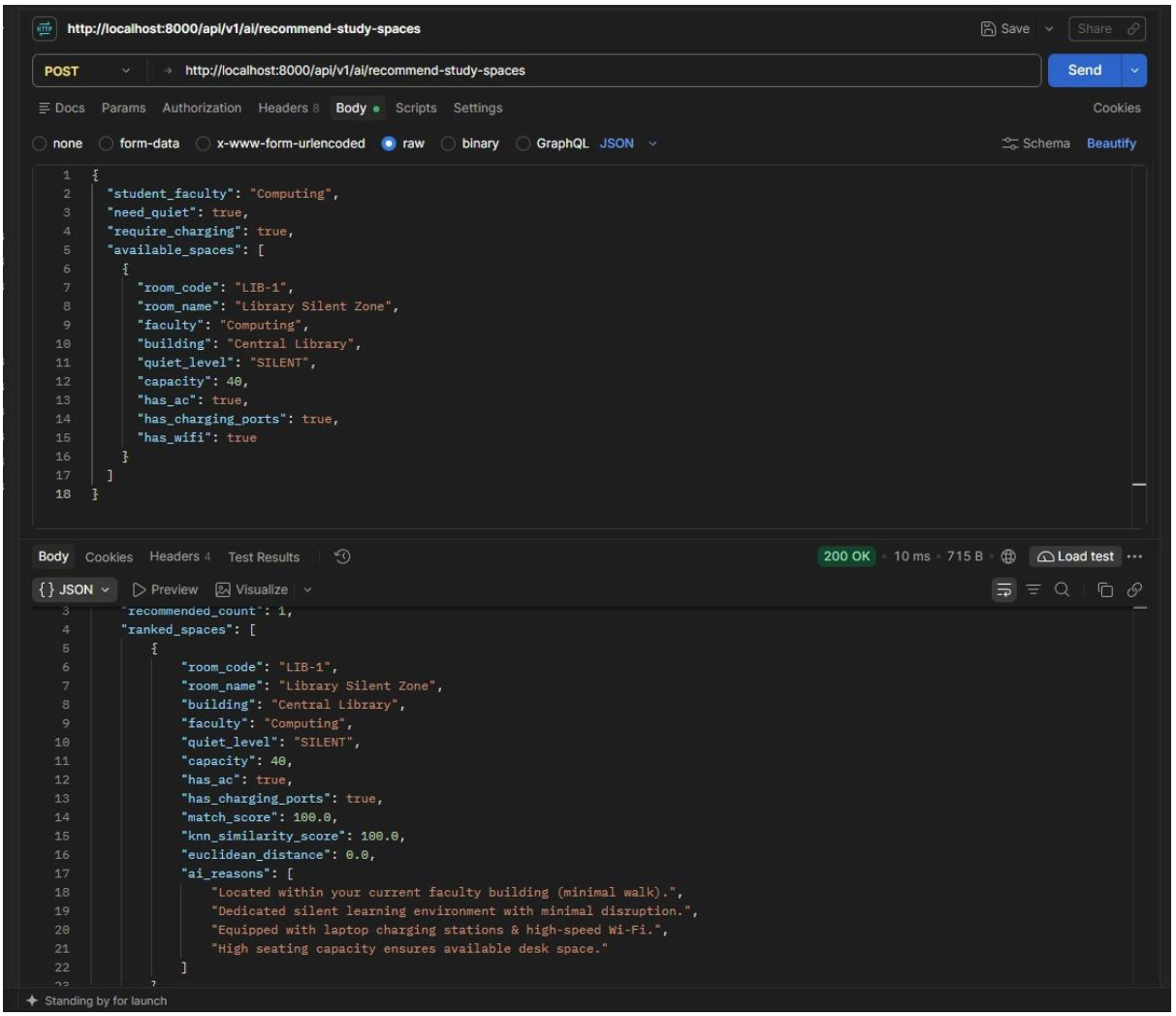
The reported configuration consists of:

Component	Configuration
Optimization Method	Genetic Algorithm
Population Size	30
Number of Generations	40
Selection	Tournament Selection
Crossover	Uniform Crossover
Mutation	Random Mutation
Fitness	Utilization Score − Penalties − Distance Penalty

Tournament selection is used to select candidate solutions, while uniform crossover combines information from selected solutions. Random mutation introduces small changes to maintain diversity in the population.

The fitness function is designed to reward better classroom utilization while applying penalties for constraint violations and unnecessary distance.

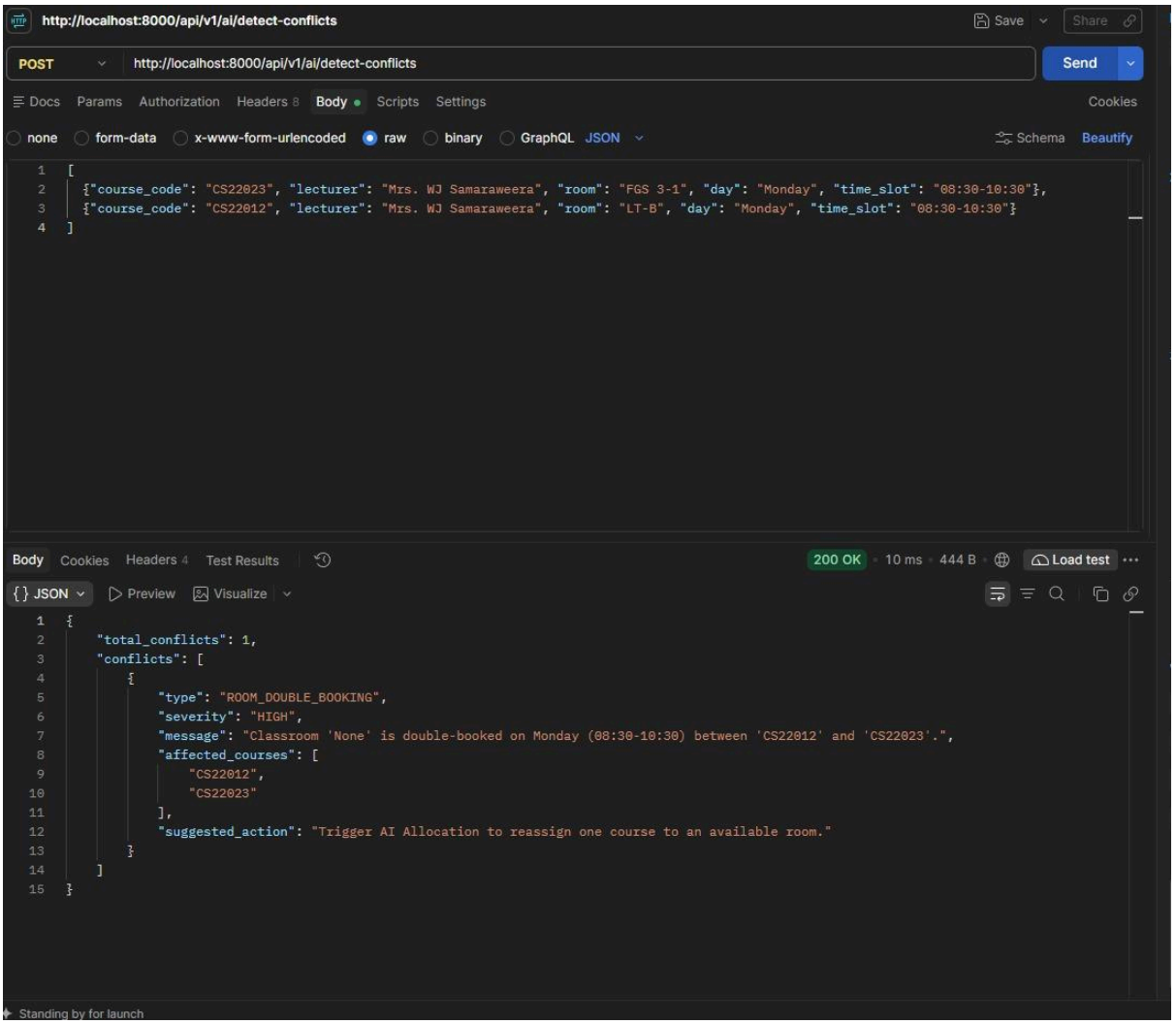
9.3 Study-Space Recommendation



The study-space recommendation component identifies suitable vacant spaces based on available information and user requirements.

MAUT is used to evaluate multiple criteria, while Content-Based Filtering supports the selection of spaces that are more relevant to the user's requirements.

9.4 Emergency Classroom Reallocation



When a classroom becomes unavailable, the system identifies alternative classrooms based on available capacity, timetable requirements, location, and other relevant constraints.

This allows the system to respond to unexpected classroom changes without requiring the entire allocation process to be performed manually.

10. Testing and Results

10.1 Test Environment

Component	Testing Environment
Operating System	Windows 10/11
Database	MySQL 8.0
Database Management	MySQL Workbench
AI Service	Python + FastAPI
AI Service Port	8000
Backend	Java Spring Boot
Frontend	React.js
Frontend Development Toot	Vite
API Testing	Swagger UI/Postman
Browser	Google Chrome
Machine Learning	Python, Scikit-Learn, Pandas, NumPy

10.2 Attendance Prediction Model Testing

Testing Method

The attendance prediction component was evaluated using the test portion of the prepared dataset. The Random Forest Regressor was trained using the training data and then evaluated using previously unseen test records.

The following metrics were used:

- R^2 Score
- Mean Absolute Error (MAE)

The R^2 score was used to measure how well the model explains the variation in attendance values, while MAE was used to measure the average difference between predicted and actual attendance.

Attendance Prediction Results

The developed Random Forest model achieved the following results:

Evaluation Metric	Result
R^2 Score	0.995
Mean Absolute Error	1.45 students

The R^2 score of **0.9951** indicates that the model achieved a strong predictive performance on the evaluated test dataset.

The MAE of **1.45 students** indicates that the predicted attendance differed from the actual attendance by approximately 1–2 students on average.

Therefore, the model provided a suitable prediction component for supporting the classroom allocation process.

Classroom Optimization Testing

The classroom optimization component was tested to determine whether the Genetic Algorithm could generate suitable classroom allocations while satisfying the defined scheduling constraints.

The optimizer was evaluated using a predefined set of courses, sessions, classrooms, capacities, and scheduling requirements.

The following measures were considered:

- Classroom utilization
- Number of room clashes
- Number of lecturer clashes
- Number of intake/batch clashes
- Number of overcrowded sessions
- Unused classroom capacity
- Optimization execution time
- Final fitness value

How to Test the Genetic Algorithm

Experiment 1 - Normal Scheduling

Use your normal dataset Record:

- Number of session
- Number of classrooms
- Number of students
- Number of constraints
- Best fitness score
- Contraint violations
- Utilization
- Execution time

Experiment 2 - Increased Number of sessions

Test	Sessions
Test 1	50
Test 2	100
Test 3	200
Test 4	300
Test 5	500

Genetic Algorithm Parameter Testing

Experiment	Population	Mutation Rate	Generations
A	50	0.05	20
B	100	0.05	40
C	150	0.10	40
D	200	0.10	60

Functional Testing

Test ID	Function	Test Input/Action	Expected Result	Actual Result	Status
FT - 01	Login	Valid credentials	User logged in	As expected	Pass
FT - 02	Attendance Prediction	Valid session data	Attendance predicted	As expected	Pass
FT - 03	Classroom Allocation	Valid timetable	Suitable room assigned	As expected	Pass
FT - 04	Clash Detection	Overlapping sessions	Clash identified	As expected	Pass
FT - 05	Study Space	Student preferences	Suitable space recommended	As expected	Pass
FT - 06	Emergency Reallocation	Room unavailable	Alternative room suggested	As expected	Pass
FT - 07	XAI	Attendance prediction	Explanation displayed	As expected	Pass
FT - 08	Timetable	Valid schedule	Timetable displayed	As expected	Pass

11. Limitations and Discussions

While the proposed system can be used to manage the classroom intelligently, there are some limitations to consider

The accuracy of the attendance prediction depends on the quality and representativeness of the available data. The accuracy of the predictions may be reduced if the historical attendance does not reflect the future behaviour of the students.

There are also a number of conflicting goals in the classroom allocation problem. As an example, optimizing the use of the classroom space may sometimes come at the cost of reducing travelling distance or increasing flexibility in lesson schedules.

Another restriction is that it is currently geared towards the Faculty of Computing and not towards the university as a whole. Thus, the findings in this study may not be directly applicable to the needs of other faculties in the classroom.

The distance optimization accuracy also relies on the accuracy of the location and walking distance data in the system.

Despite these constraints, the proposed architecture can be readily scaled with more data, more university departments and more sophisticated optimization methods.

12. Ethical and Social Considerations

It is important to implement and use the system responsibly as it comes with information related to students, attendance, academic sessions and University resources.

Student information should be kept secure and only accessed by authorised users. The system should not expose any personal information in dashboards/reports that it does not need to.

Predictions of attendance should be viewed as a decision support tool, and not as a representation of each student's actual attendance.The Predictions shall not be used to unfairly punish pupils.

Transparency is also a key factor. The system can also integrate the Explainable AI feature, which will enable it to give the user understandable information about the factors influencing the important predictions and recommendations.

13. Conclusion

This AI-powered University Classroom Utilization Optimizer system aims to solve the real-world challenges of classroom allocation and utilization in universities.

The proposed system integrates machine learning, optimization, recommendation, decision making, and explainable AI techniques to offer a comprehensive classroom management solution. The random forest is employed for predicting the expected attendance and the genetic algorithm is employed for optimizing the allocation of classrooms based on utilization, constraints and physical distance.

The system is more flexible than the traditional fixed classroom allocation system because it can give suggestions for study space and can be used to rearrange classrooms in case of emergency.

The machine learning part of the system is shown to be effective in attendance prediction, and the whole system architecture proves to be a practical basis for the implementation of AI-supported classroom management.

The system can be expanded and scaled in future with increasing real-life attendance data, more faculties, more detailed stress testing and optimization of the AI models. These enhancements can benefit the system to be potentially expanded to a platform for classroom resource management across the entire university.

14. References

1. [1] L. Breiman, "Random Forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

2. [2] Y.-Z. Wang, "Using genetic algorithm methods to solve course scheduling problems," *Expert Syst. Appl.*, vol. 25, no. 1, pp. 39–50, 2003, doi: 10.1016/S0957-4174(03)00004-6.

3. [3] Y.-Z. Wang, "An application of genetic algorithm methods for teacher assignment problems," *Expert Syst. Appl.*, vol. 22, no. 4, pp. 295–302, 2002, doi: 10.1016/S0957-4174(02)00017-9.

4. [4] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.

5. [5] P. Lops, M. de Gemmis, and G. Semeraro, "Content-Based Recommender Systems: State of the Art and Trends," in *Recommender Systems Handbook*, Springer, 2015.

6. [6] R. L. Keeney and H. Raiffa, *Decisions with Multiple Objectives: Preferences and Value Tradeoffs*. Cambridge, U.K.: Cambridge University Press, 1993.

7. [7] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

8. [8] React Team, "React Documentation," [Online]. Available: https://react.dev/

9. [9] Spring Team, "Spring Boot Documentation," [Online]. Available: https://docs.spring.io/spring-boot/

10. [10] S. Ramírez, "FastAPI Documentation," [Online]. Available: https://fastapi.tiangolo.com/

11. [11] Oracle Corporation, "MySQL 8.0 Reference Manual," [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

15.Individual Contribution Report

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**H.A.I.D.Harshani**	**Registration No.**	**D/BIT/24/0044**
Degree / Intake	**Information Technology** **41**	**Group No.**	**31**
Project Title	**AI-Powered University Classroom Utilization Optimizer**		

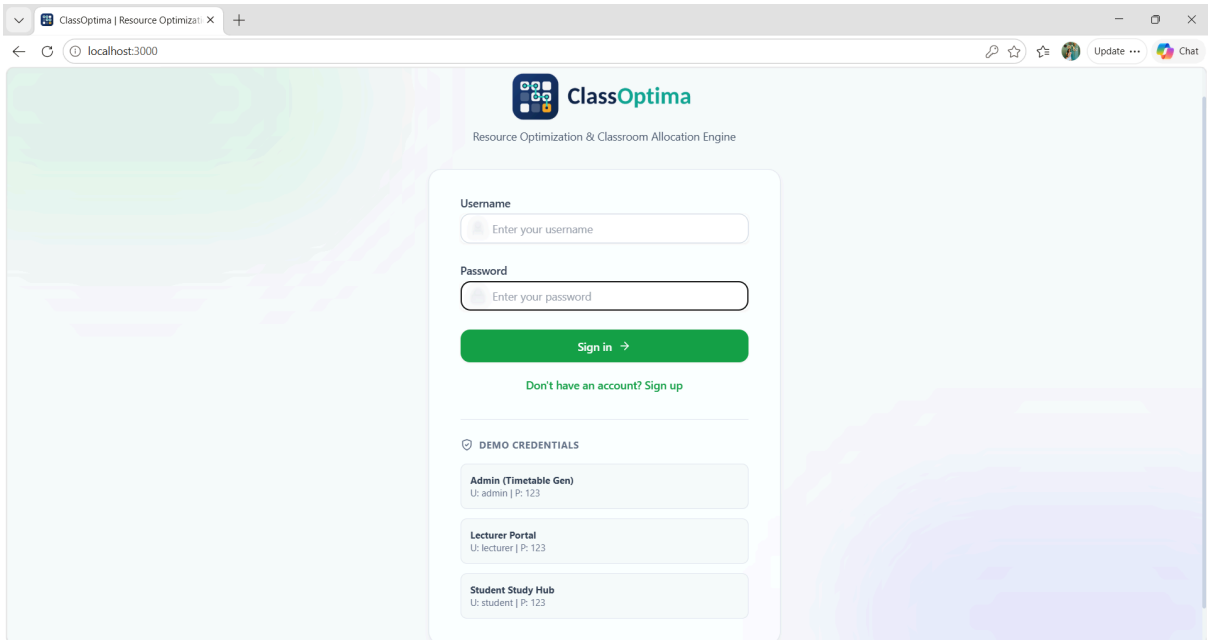
Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used
Development of All User Dashboards (Admin, Lecturer, and Student)	Designed and developed the Admin, Lecturer, and Student dashboards with role-specific interfaces, organized layouts, navigation, and relevant features to provide users with easy access to classroom, timetable, and system information.	● React.js ● JavaScript ● Tailwind CSS ● Lucide React ● Component-Based Development ● Responsive UI Design
Campus map interface	Developed the frontend interface for displaying campus locations and helping users identify university buildings and classroom locations.	● React.js ● JavaScript ● Tailwind CSS map component/library
Heuristic search for classroom allocation	Contributed to the use of heuristic search techniques to evaluate possible classroom allocations based on relevant criteria and prioritize suitable options such as classroom capacity, availability, required facilities, and timetable	● Heuristic Search Algorithm, constraint-based evaluation

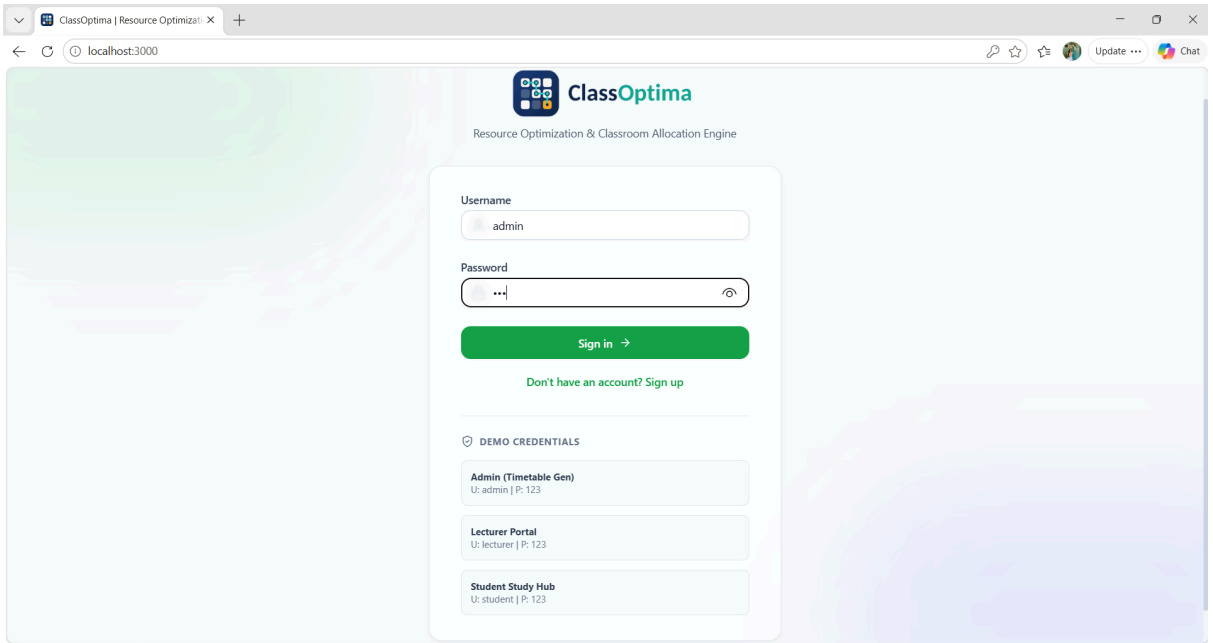
	conflicts when determining classroom suitability.	
API Integration and Frontend–Backend Communication	Integrated Axios-based HTTP requests to communicate with backend APIs and retrieve relevant system information for the frontend.	● REST APIs(Postman) ● Axios ● JSON ● JavaScipt
Classroom Constraint Evaluation	Contributed to considering classroom capacity, availability, required facilities, and timetable conflicts when determining classroom suitability.	● Constraint-based search ● heuristic evaluation
Frontend State Management and Local Data Handling	Managed frontend state and handled local data to support user interactions and display relevant information across the dashboards	● React Hooks ● JavaScript
Integration of Classroom Optimization Results	Integrated and displayed classroom optimization results in the frontend so users could view the suggested classroom allocations.	● React.js ● REST APIs ● JSON ● Frontend–Backend Integration

Evidence/Output

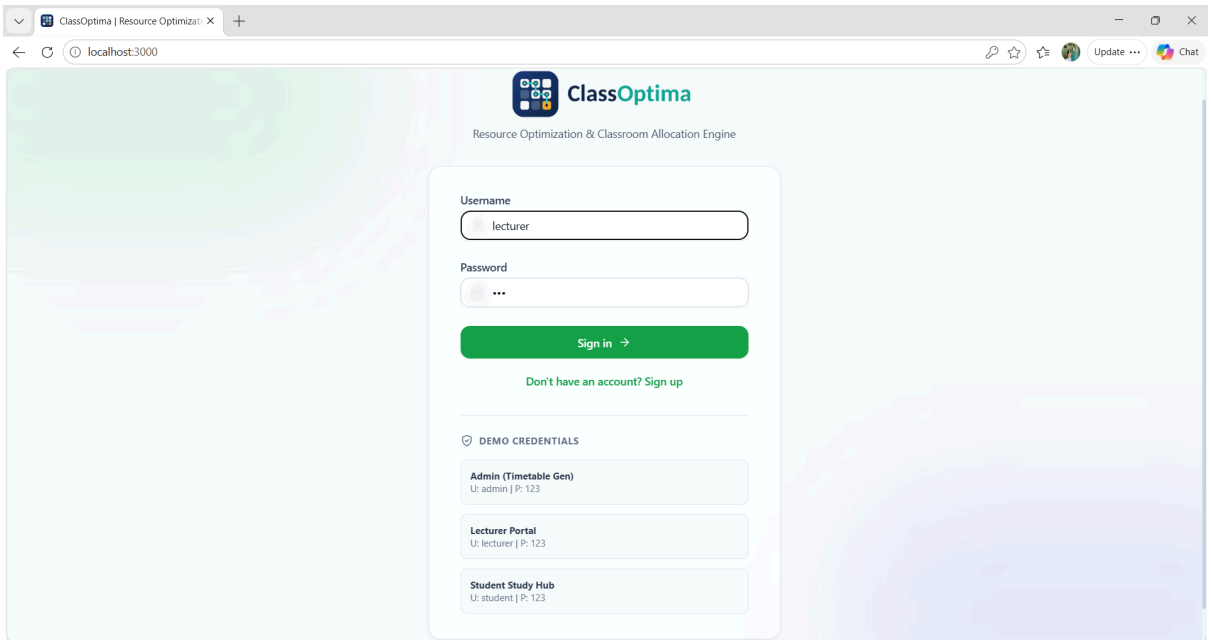
Frontend Architecture and Development



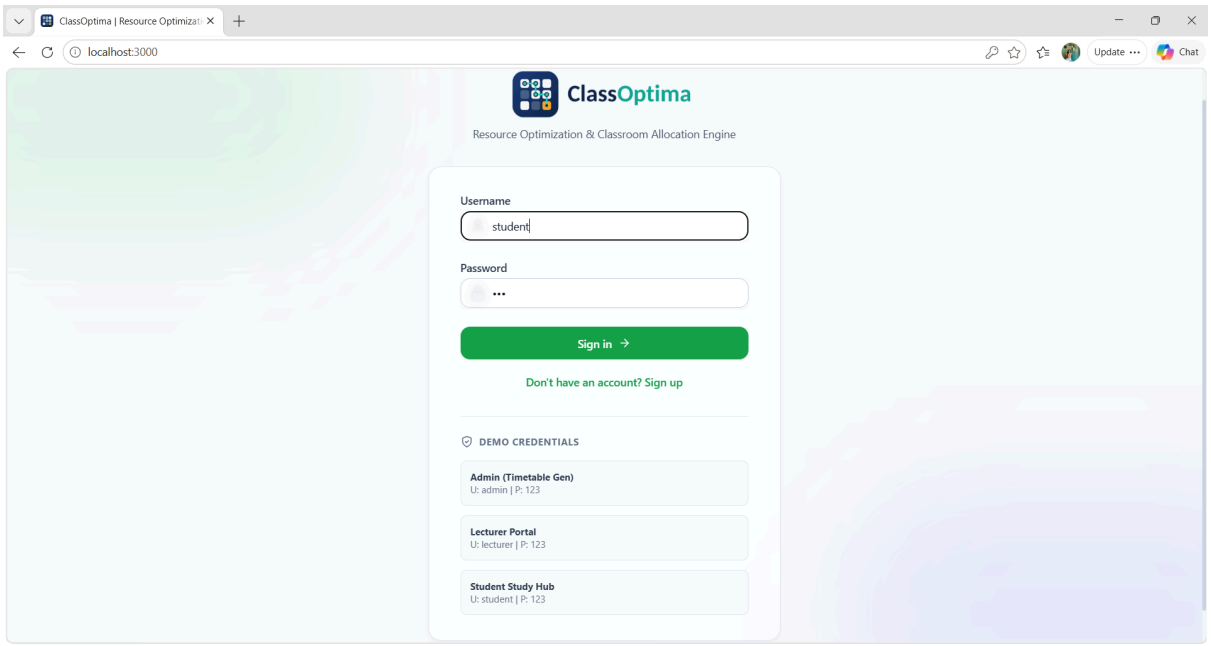
Admin dashboard

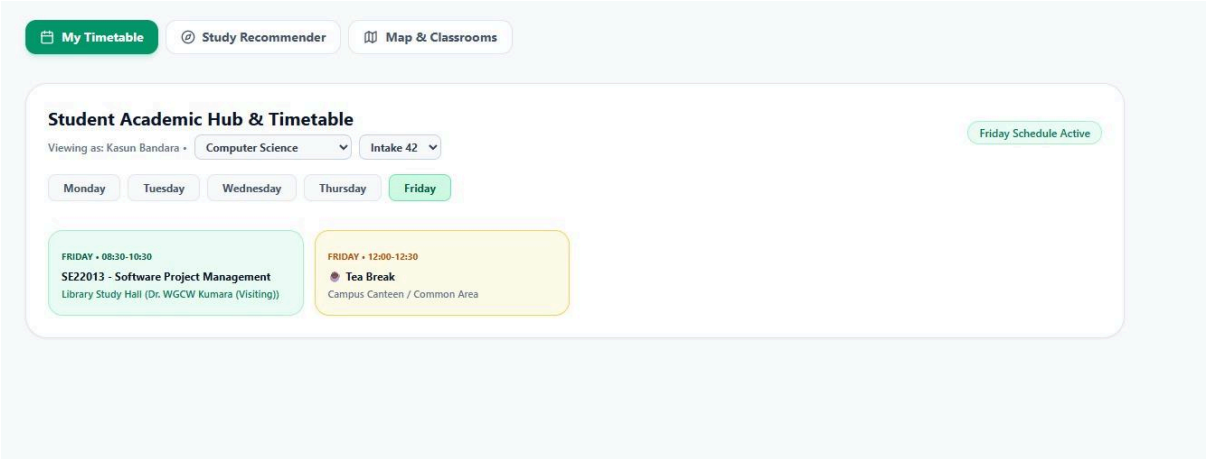


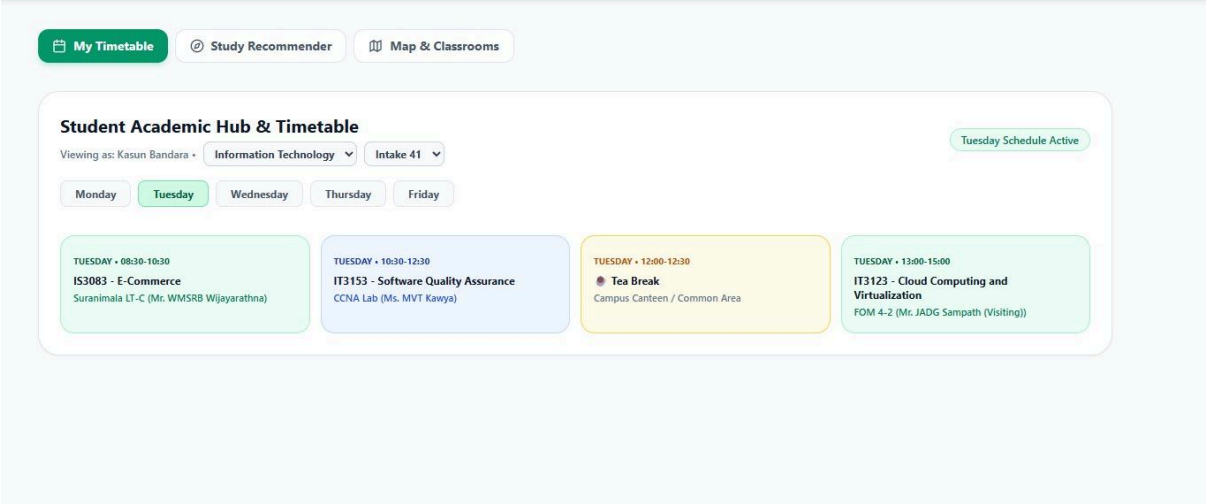
Lecturer portal



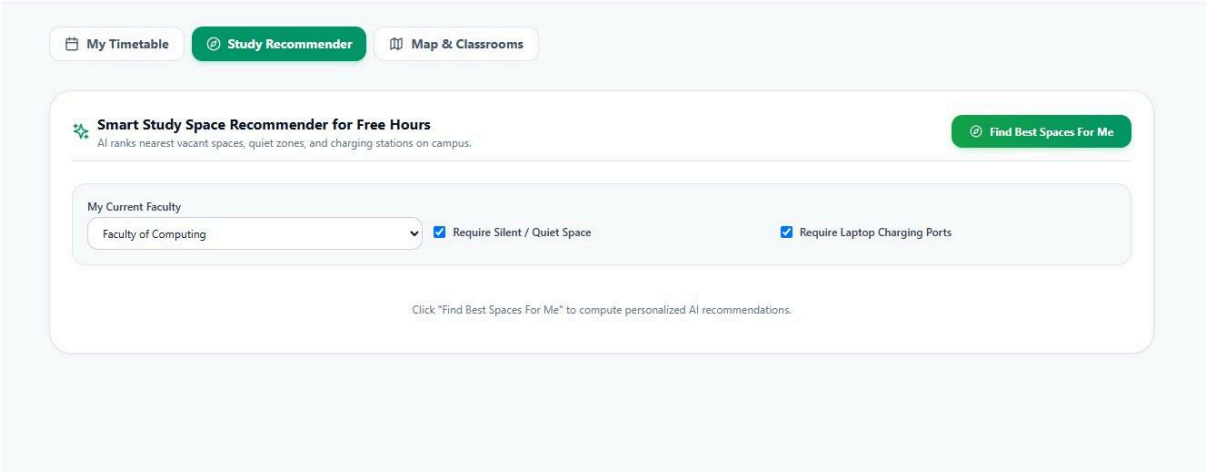
Student Portal

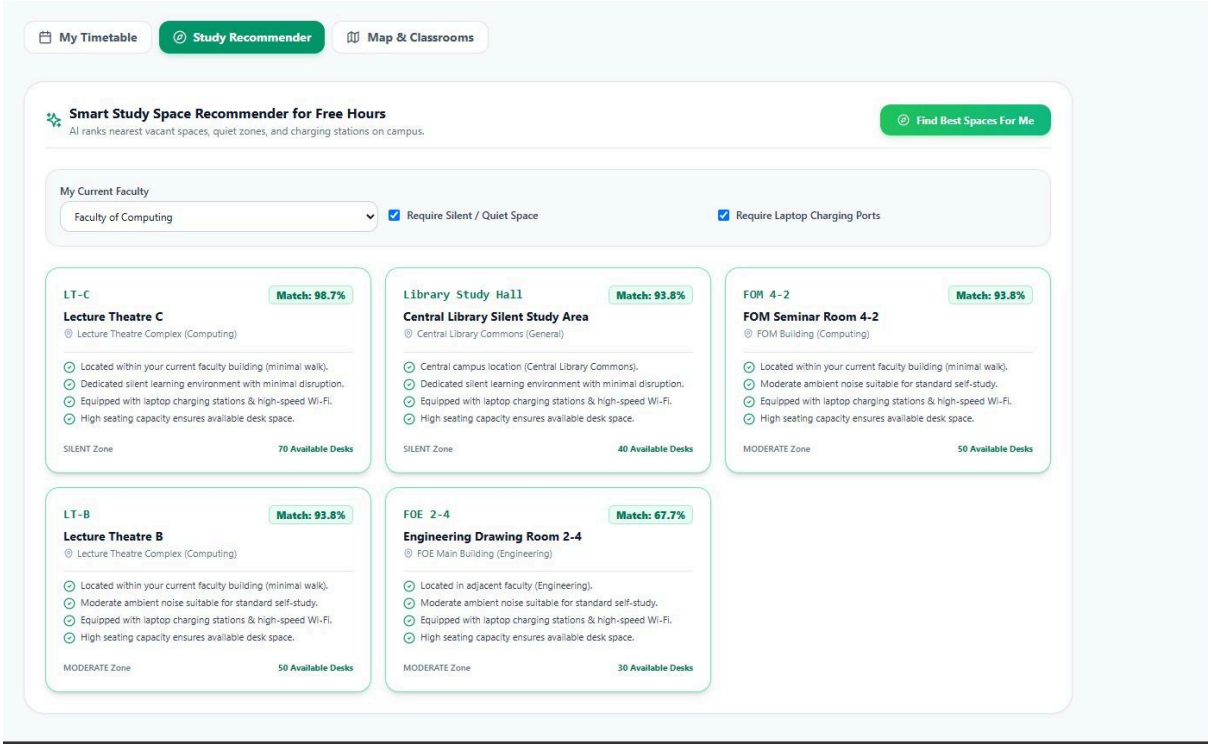




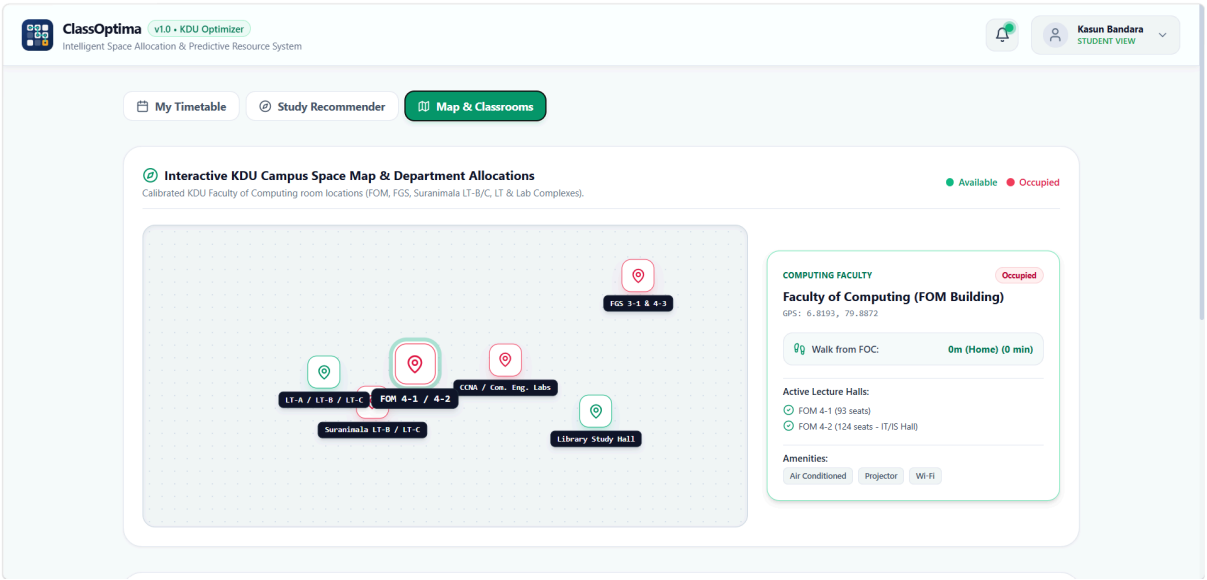


Heuristic Search Approach

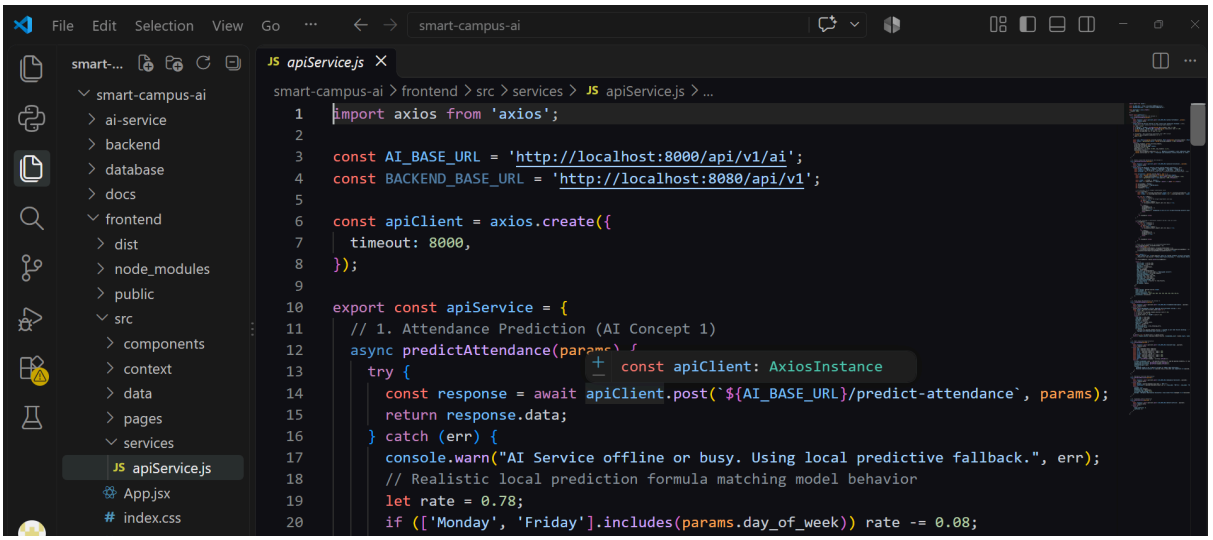




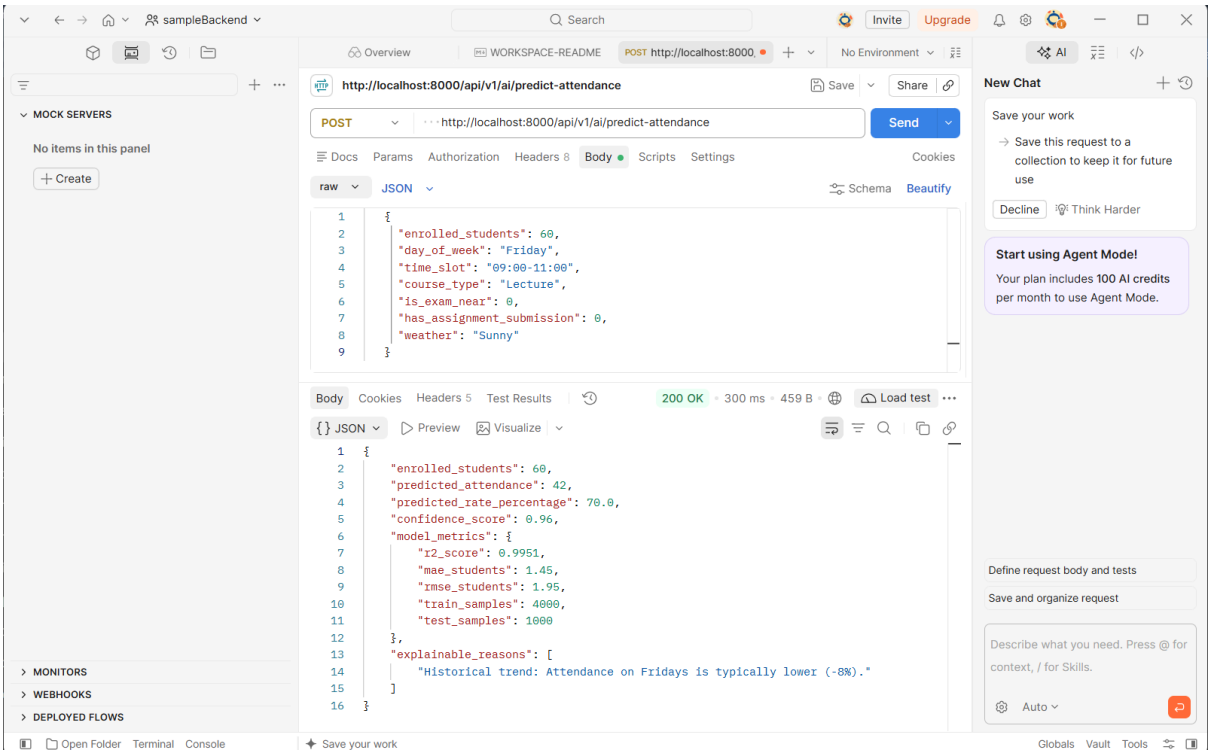
Campus map interface



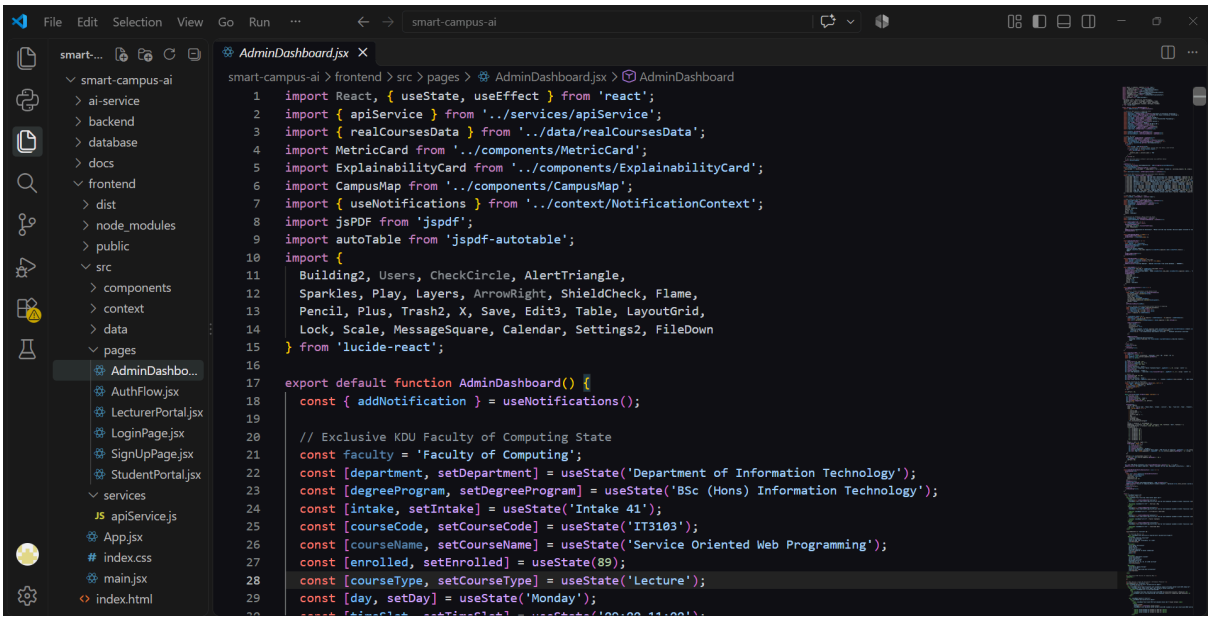
API Integration and Frontend–Backend Communication

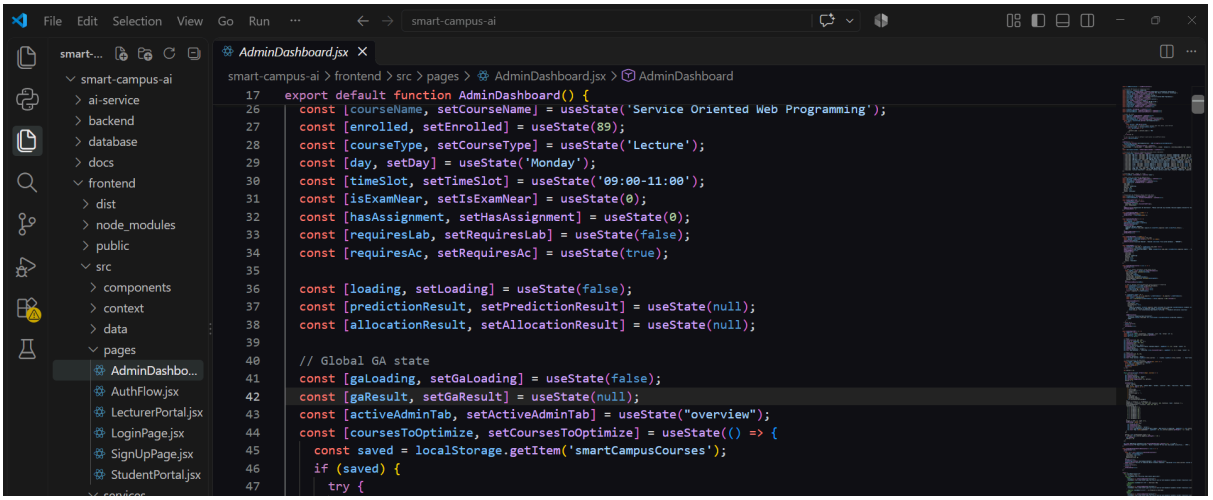


API response or data displayed in the React interface

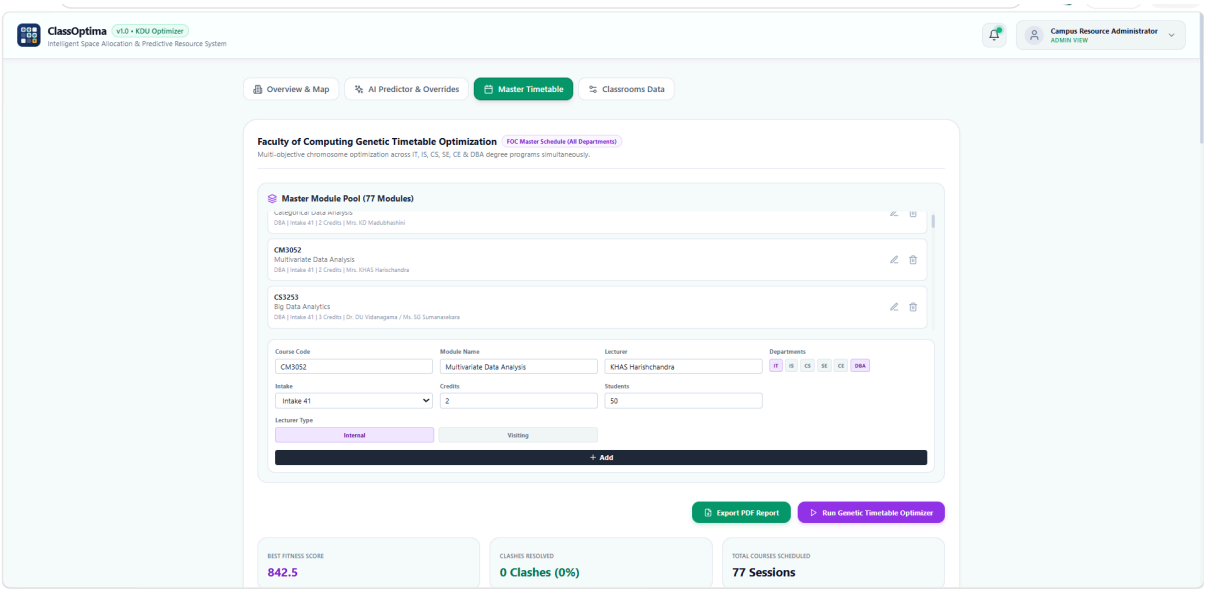


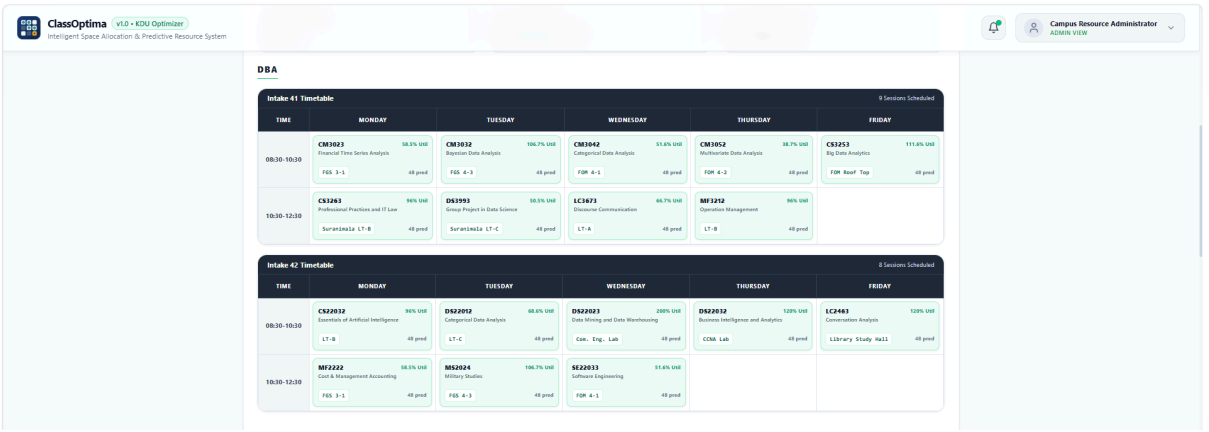
State Management and Local Data Handling



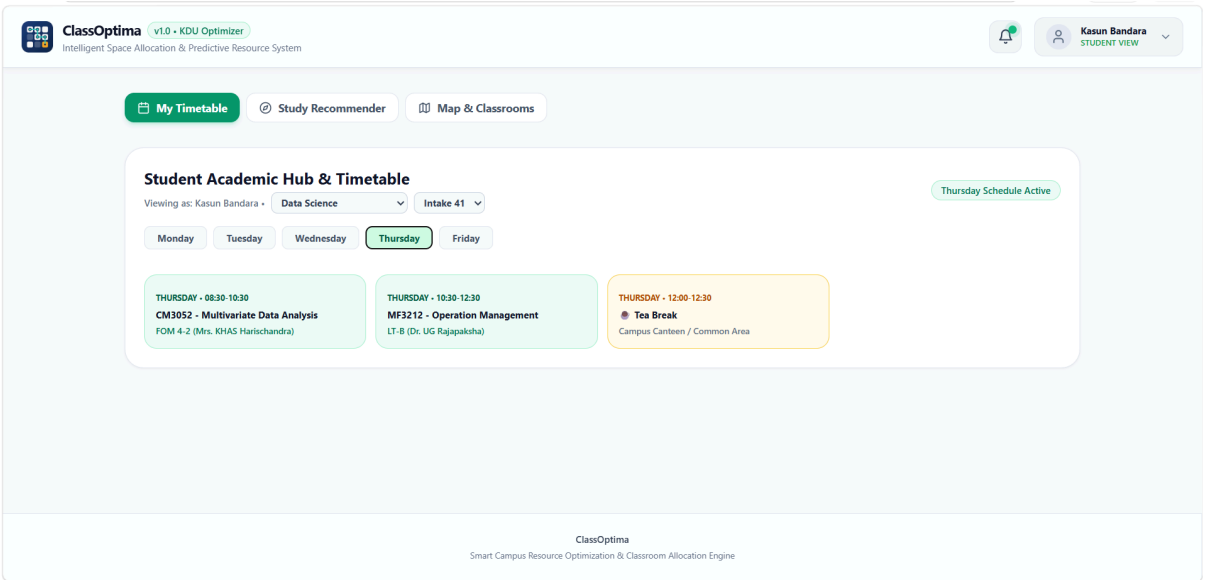


Actual frontend result showing an AI-generated allocation or timetable





Integrated Classroom Optimization Results Generated by the Genetic Timetable Optimizer



Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student HAID Harshani **Date:** 17/10/2026

Signature:

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**HDMC Nawarathna**	**Registration No.**	**D/COE/25/0023**
Degree / Intake	**COE / 42**	**Group No.**	**31**
Project Title	**AI-Powered University Classroom Utilization Optimizer**		

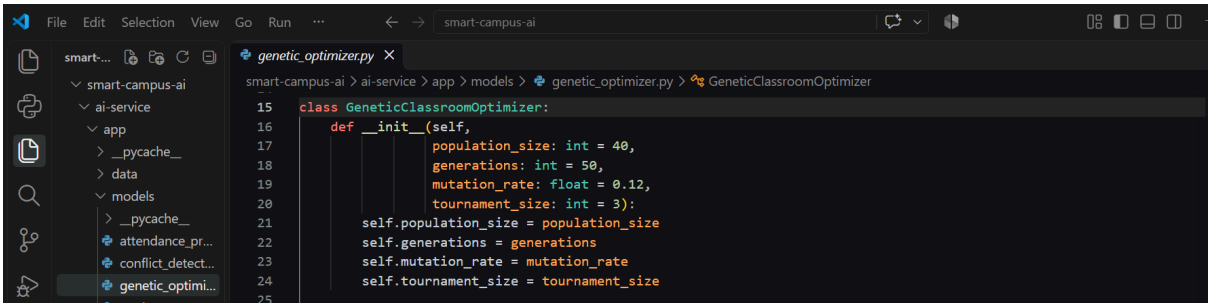
Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used
Genetic Algorithm timetable optimization	Implemented the GeneticClassroomOptimizer class. Each timetable is a chromosome in which every gene holds a course, a room, a day and a time slot. I built the random population initialization, which prefers lab rooms for lab courses and picks time slots by course credits. I also implemented tournament selection, uniform crossover, random room and slot mutation, and elitism that keeps the top 2 individuals in every generation.	● Python; Genetic Algorithm (evolutionary search); tournament selection; uniform crossover; mutation; elitism
Constraint validation	Coded the constraint checks for room clashes, lecturer clashes and batch (intake) clashes. I also coded room capacity against predicted attendance, and the rule that lab courses must not be placed in standard lecture halls.	● Constraint-based evaluation inside the GA; Python
Fitness function design	Designed the scoring model. Hard penalties apply to clashes (-1500 each) and to a lab course in a standard hall (-1200). A capacity shortfall is penalized in proportion to the deficit. Soft rewards go to good seat utilization, AC rooms, home-faculty buildings and efficient use of seats.	● Multi-objective fitness function; weighted penalties and rewards

FastAPI optimization endpoint	Built the POST /api/v1/ai/optimize-allocations endpoint. It accepts courses and classrooms, runs the GA, and returns the allocations, total clashes, a clash breakdown, the best fitness score and the fitness convergence history.	• FastAPI; Pydantic schemas; REST API; Swagger UI
Integration with attendance prediction	Connected the Random Forest attendance model to the optimizer. Predicted attendance is calculated for any course that doesn't have it, then used by the fitness function to size rooms and calculate utilization.	• Random Forest predictions (scikit-learn); Python service integration
Algorithm tuning and performance	Tuned the GA parameters: population size, number of generations, mutation rate (0.12) and tournament size (3). I tracked the best fitness per generation to check that the search converges and that clashes are reduced.	• Parameter tuning; convergence analysis
Fallback and reliability	Built the frontend fallback so the Admin Dashboard still produces a clash-aware timetable when the AI service is offline. It avoids slot clashes for the same intake and department.	• JavaScript (React); Axios error handling; heuristic scheduling
Final timetable generation	Delivered the end-to-end result in the Admin Dashboard: a generated timetable with room assignments, the best fitness score, clashes resolved and total sessions scheduled.	• React frontend; FastAPI; Genetic Algorithm

Evidence/Output

GA setup and population initialization



```python
class GeneticClassroomOptimizer:
    def __init__(self,
                 population_size: int = 40,
                 generations: int = 50,
                 mutation_rate: float = 0.12,
                 tournament_size: int = 3):
        self.population_size = population_size
        self.generations = generations
        self.mutation_rate = mutation_rate
        self.tournament_size = tournament_size
```

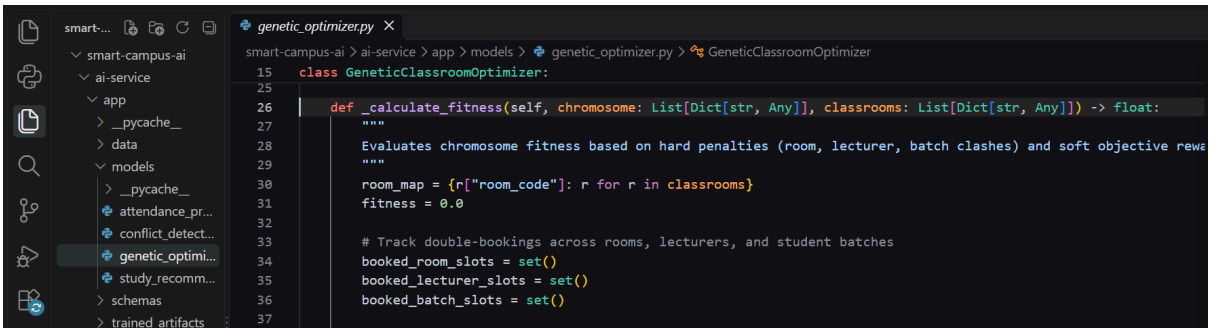


```python
class GeneticClassroomOptimizer:
    def _create_individual(self, courses: List[Dict[str, Any]], classrooms: List[Dict[str, Any]]) -> List[Dict[str,
        individual = []
        for course in courses:
            # Filter candidate rooms roughly matching lab constraint if possible
            req_lab = course.get("requires_lab", False)
            matching_rooms = [r for r in classrooms if r.get("is_lab", False) == req_lab]
            if not matching_rooms:
                matching_rooms = classrooms

            chosen_room = random.choice(matching_rooms)["room_code"]
            credits = course.get("credits", 2)
            slots = SLOTS_3_CREDIT if credits >= 3 else SLOTS_2_CREDIT
            chosen_day = random.choice(DAYS)
            chosen_slot = random.choice(slots)

            individual.append({
                "course": course,
                "assigned_room": chosen_room,
                "assigned_day": chosen_day,
                "assigned_time_slot": chosen_slot
            })
        return individual
```

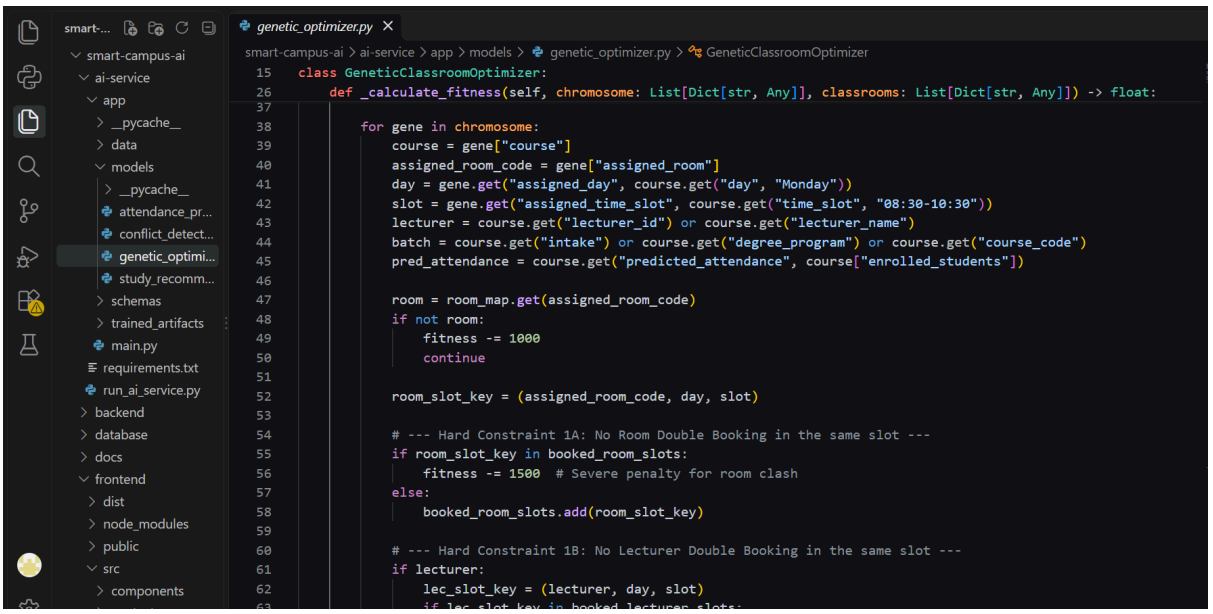
Constraint checking

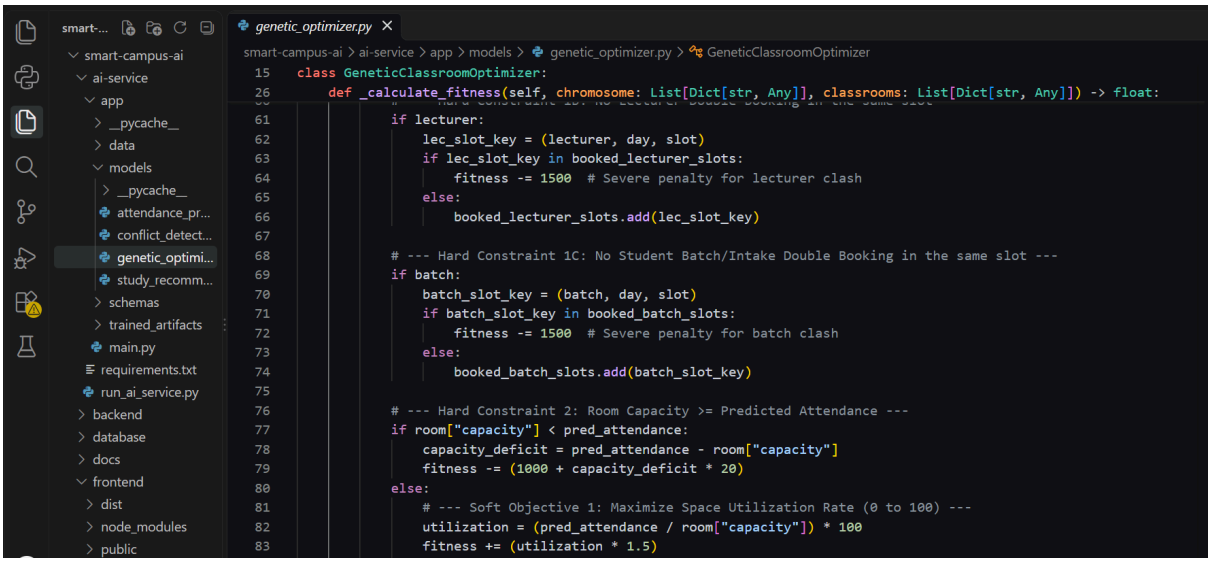


```python
class GeneticClassroomOptimizer:

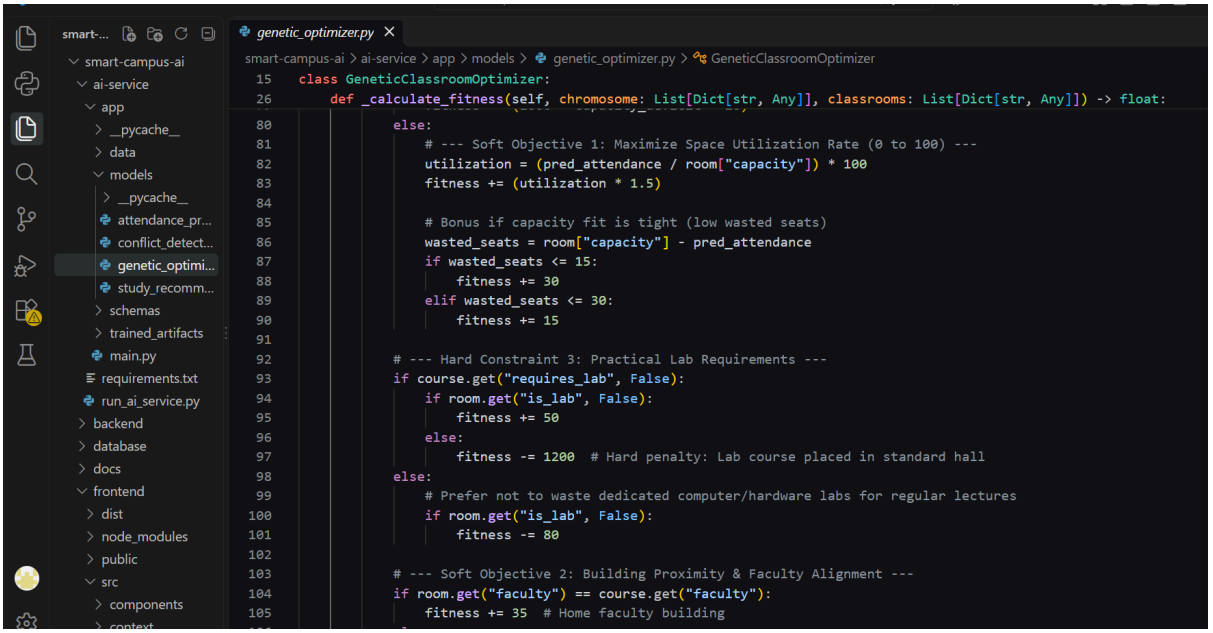
    def _calculate_fitness(self, chromosome: List[Dict[str, Any]], classrooms: List[Dict[str, Any]]) -> float:
        """
        Evaluates chromosome fitness based on hard penalties (room, lecturer, batch clashes) and soft objective rewa
        """
        room_map = {r["room_code"]: r for r in classrooms}
        fitness = 0.0

        # Track double-bookings across rooms, lecturers, and student batches
        booked_room_slots = set()
        booked_lecturer_slots = set()
        booked_batch_slots = set()
```

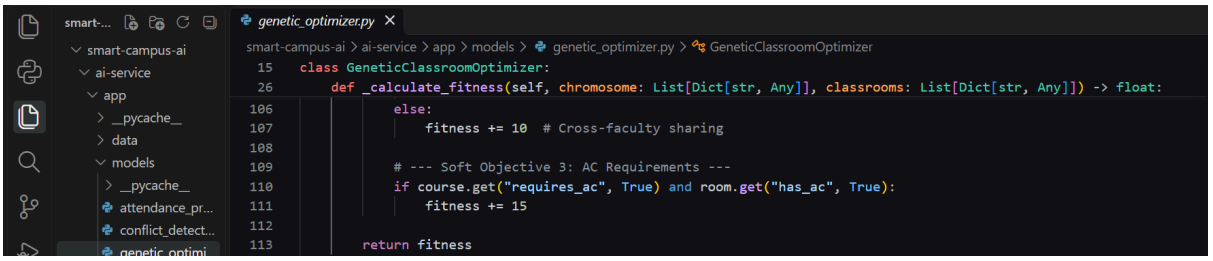

smart-...
genetic_optimizer.py
smart-campus-ai > ai-service > app > models > genetic_optimizer.py > GeneticClassroomOptimizer
smart-campus-ai
ai-service
app
__pycache__
data
models
__pycache__
attendance_pr...
conflict_detect...
genetic_optimi...
study_recomm...
schemas
trained_artifacts
main.py
requirements.txt
run_ai_service.py
backend
database
docs
frontend
dist
node_modules
public
src
components
15 class GeneticClassroomOptimizer:
26 def _calculate_fitness(self, chromosome: List[Dict[str, Any]], classrooms: List[Dict[str, Any]]) -> float:
38 for gene in chromosome:
39 course = gene["course"]
40 assigned_room_code = gene["assigned_room"]
41 day = gene.get("assigned_day", course.get("day", "Monday"))
42 slot = gene.get("assigned_time_slot", course.get("time_slot", "08:30-10:30"))
43 lecturer = course.get("lecturer_id") or course.get("lecturer_name")
44 batch = course.get("intake") or course.get("degree_program") or course.get("course_code")
45 pred_attendance = course.get("predicted_attendance", course["enrolled_students"])
47 room = room_map.get(assigned_room_code)
48 if not room:
49 fitness -= 1000
50 continue
52 room_slot_key = (assigned_room_code, day, slot)
54 # --- Hard Constraint 1A: No Room Double Booking in the same slot ---
55 if room_slot_key in booked_room_slots:
56 fitness -= 1500 # Severe penalty for room clash
57 else:
58 booked_room_slots.add(room_slot_key)
60 # --- Hard Constraint 1B: No Lecturer Double Booking in the same slot ---
61 if lecturer:
62 lec_slot_key = (lecturer, day, slot)


smart-...
genetic_optimizer.py
smart-campus-ai > ai-service > app > models > genetic_optimizer.py > GeneticClassroomOptimizer
smart-campus-ai
ai-service
app
__pycache__
data
models
__pycache__
attendance_pr...
conflict_detect...
genetic_optimi...
study_recomm...
schemas
trained_artifacts
main.py
requirements.txt
run_ai_service.py
backend
database
docs
frontend
dist
node_modules
public
15 class GeneticClassroomOptimizer:
26 def _calculate_fitness(self, chromosome: List[Dict[str, Any]], classrooms: List[Dict[str, Any]]) -> float:
61 if lecturer:
62 lec_slot_key = (lecturer, day, slot)
63 if lec_slot_key in booked_lecturer_slots:
64 fitness -= 1500 # Severe penalty for lecturer clash
65 else:
66 booked_lecturer_slots.add(lec_slot_key)
68 # --- Hard Constraint 1C: No Student Batch/Intake Double Booking in the same slot ---
69 if batch:
70 batch_slot_key = (batch, day, slot)
71 if batch_slot_key in booked_batch_slots:
72 fitness -= 1500 # Severe penalty for batch clash
73 else:
74 booked_batch_slots.add(batch_slot_key)
76 # --- Hard Constraint 2: Room Capacity >= Predicted Attendance ---
77 if room["capacity"] < pred_attendance:
78 capacity_deficit = pred_attendance - room["capacity"]
79 fitness -= (1000 + capacity_deficit * 20)
80 else:
81 # --- Soft Objective 1: Maximize Space Utilization Rate (0 to 100) ---
82 utilization = (pred_attendance / room["capacity"]) * 100
83 fitness += (utilization * 1.5)

Fitness function (penalties and rewards)

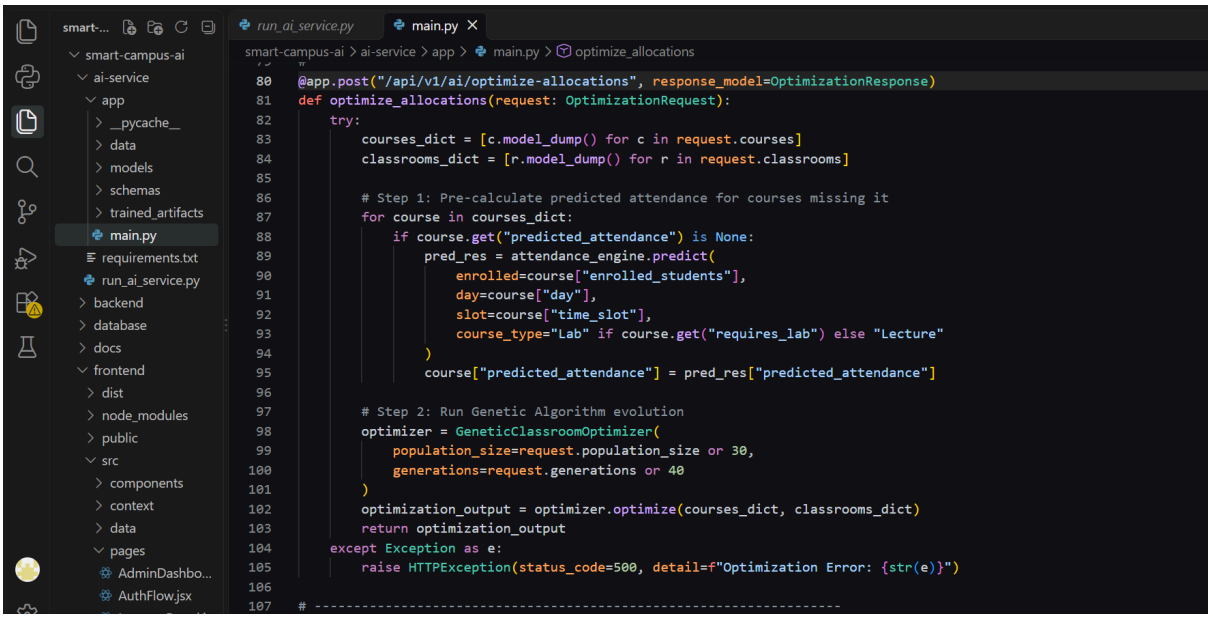


```
genetic_optimizer.py
smart-campus-ai > ai-service > app > models > genetic_optimizer.py > GeneticClassroomOptimizer
15  class GeneticClassroomOptimizer:
26      def _calculate_fitness(self, chromosome: List[Dict[str, Any]], classrooms: List[Dict[str, Any]]) -> float:
80              else:
81                  # --- Soft Objective 1: Maximize Space Utilization Rate (0 to 100) ---
82                  utilization = (pred_attendance / room["capacity"]) * 100
83                  fitness += (utilization * 1.5)
84
85                  # Bonus if capacity fit is tight (low wasted seats)
86                  wasted_seats = room["capacity"] - pred_attendance
87                  if wasted_seats <= 15:
88                      fitness += 30
89                  elif wasted_seats <= 30:
90                      fitness += 15
91
92              # --- Hard Constraint 3: Practical Lab Requirements ---
93              if course.get("requires_lab", False):
94                  if room.get("is_lab", False):
95                      fitness += 50
96                  else:
97                      fitness -= 1200  # Hard penalty: Lab course placed in standard hall
98              else:
99                  # Prefer not to waste dedicated computer/hardware labs for regular lectures
100                 if room.get("is_lab", False):
101                     fitness -= 80
102
103             # --- Soft Objective 2: Building Proximity & Faculty Alignment ---
104             if room.get("faculty") == course.get("faculty"):
105                 fitness += 35  # Home faculty building
```



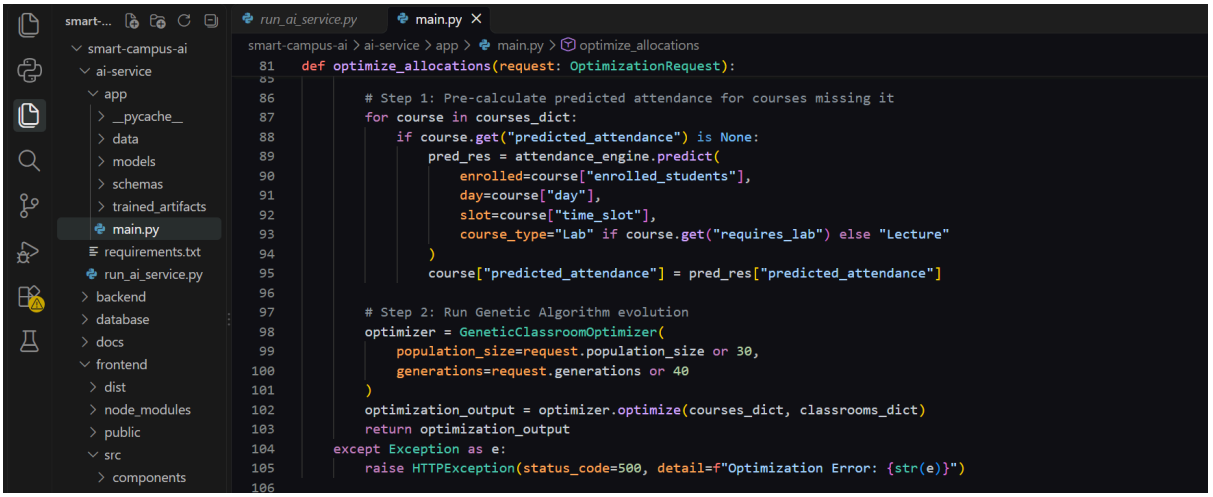
```
genetic_optimizer.py
smart-campus-ai > ai-service > app > models > genetic_optimizer.py > GeneticClassroomOptimizer
15  class GeneticClassroomOptimizer:
26      def _calculate_fitness(self, chromosome: List[Dict[str, Any]], classrooms: List[Dict[str, Any]]) -> float:
106             else:
107                 fitness += 10  # Cross-faculty sharing
108
109             # --- Soft Objective 3: AC Requirements ---
110             if course.get("requires_ac", True) and room.get("has_ac", True):
111                 fitness += 15
112
113         return fitness
```

FastAPI optimization endpoint



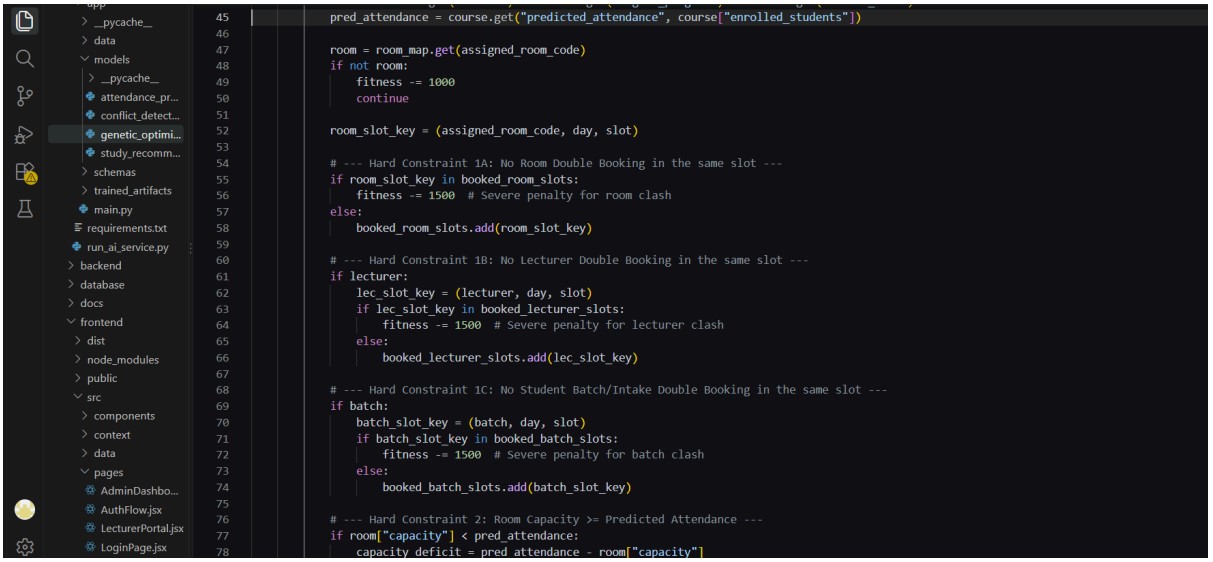
```
run_ai_service.py   main.py
smart-campus-ai > ai-service > app > main.py > optimize_allocations
80  @app.post("/api/v1/ai/optimize-allocations", response_model=OptimizationResponse)
81  def optimize_allocations(request: OptimizationRequest):
82      try:
83          courses_dict = [c.model_dump() for c in request.courses]
84          classrooms_dict = [r.model_dump() for r in request.classrooms]
85
86          # Step 1: Pre-calculate predicted attendance for courses missing it
87          for course in courses_dict:
88              if course.get("predicted_attendance") is None:
89                  pred_res = attendance_engine.predict(
90                      enrolled=course["enrolled_students"],
91                      day=course["day"],
92                      slot=course["time_slot"],
93                      course_type="Lab" if course.get("requires_lab") else "Lecture"
94                  )
95                  course["predicted_attendance"] = pred_res["predicted_attendance"]
96
97          # Step 2: Run Genetic Algorithm evolution
98          optimizer = GeneticClassroomOptimizer(
99              population_size=request.population_size or 30,
100             generations=request.generations or 40
101         )
102         optimization_output = optimizer.optimize(courses_dict, classrooms_dict)
103         return optimization_output
104     except Exception as e:
105         raise HTTPException(status_code=500, detail=f"Optimization Error: {str(e)}")
106
107 # ----------------------------------------------------------------
```

Attendance prediction connected to the optimizer



```python
def optimize_allocations(request: OptimizationRequest):
        # Step 1: Pre-calculate predicted attendance for courses missing it
        for course in courses_dict:
            if course.get("predicted_attendance") is None:
                pred_res = attendance_engine.predict(
                    enrolled=course["enrolled_students"],
                    day=course["day"],
                    slot=course["time_slot"],
                    course_type="Lab" if course.get("requires_lab") else "Lecture"
                )
                course["predicted_attendance"] = pred_res["predicted_attendance"]

        # Step 2: Run Genetic Algorithm evolution
        optimizer = GeneticClassroomOptimizer(
            population_size=request.population_size or 30,
            generations=request.generations or 40
        )
        optimization_output = optimizer.optimize(courses_dict, classrooms_dict)
        return optimization_output
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Optimization Error: {str(e)}")
```



```python
    pred_attendance = course.get("predicted_attendance", course["enrolled_students"])

    room = room_map.get(assigned_room_code)
    if not room:
        fitness -= 1000
        continue

    room_slot_key = (assigned_room_code, day, slot)

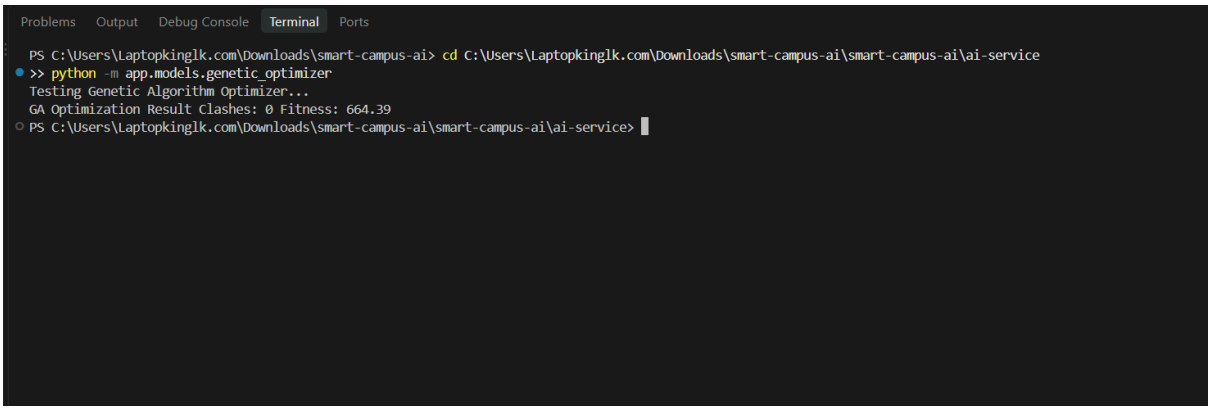
    # --- Hard Constraint 1A: No Room Double Booking in the same slot ---
    if room_slot_key in booked_room_slots:
        fitness -= 1500  # Severe penalty for room clash
    else:
        booked_room_slots.add(room_slot_key)

    # --- Hard Constraint 1B: No Lecturer Double Booking in the same slot ---
    if lecturer:
        lec_slot_key = (lecturer, day, slot)
        if lec_slot_key in booked_lecturer_slots:
            fitness -= 1500  # Severe penalty for lecturer clash
        else:
            booked_lecturer_slots.add(lec_slot_key)

    # --- Hard Constraint 1C: No Student Batch/Intake Double Booking in the same slot ---
    if batch:
        batch_slot_key = (batch, day, slot)
        if batch_slot_key in booked_batch_slots:
            fitness -= 1500  # Severe penalty for batch clash
        else:
            booked_batch_slots.add(batch_slot_key)

    # --- Hard Constraint 2: Room Capacity >= Predicted Attendance ---
    if room["capacity"] < pred_attendance:
        capacity_deficit = pred_attendance - room["capacity"]
```

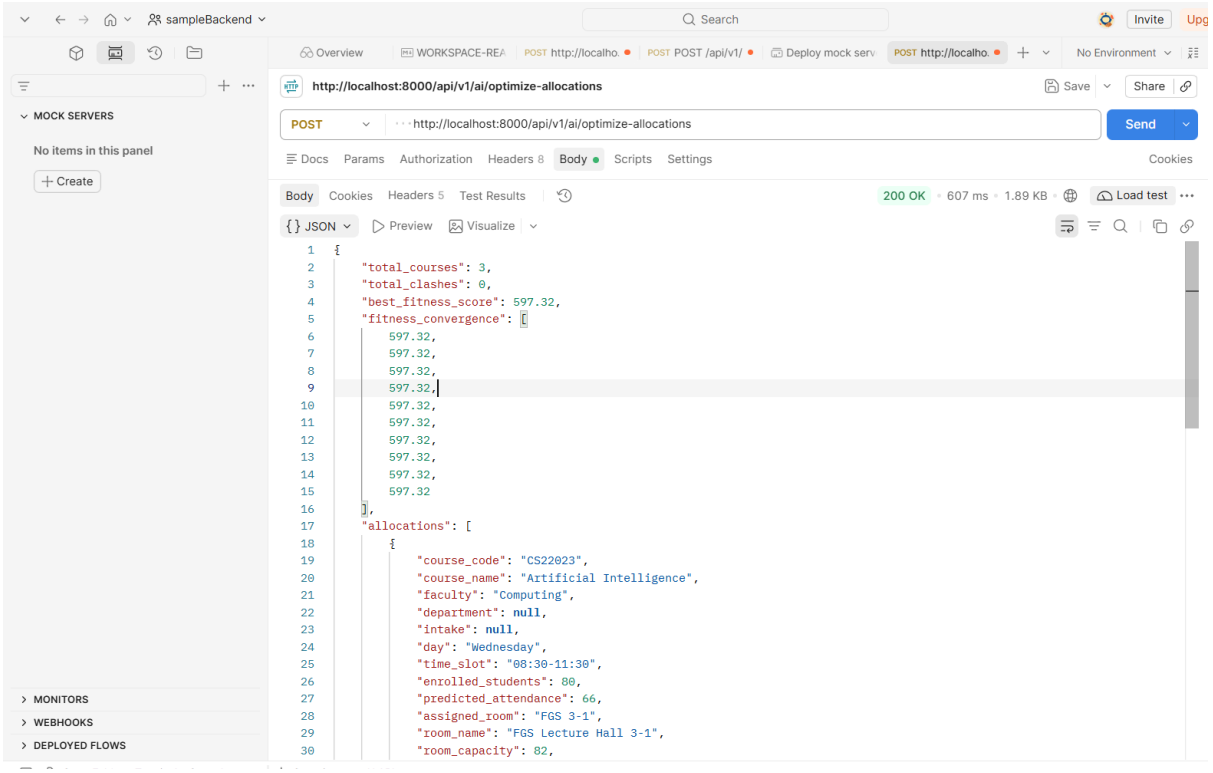
GA execution output

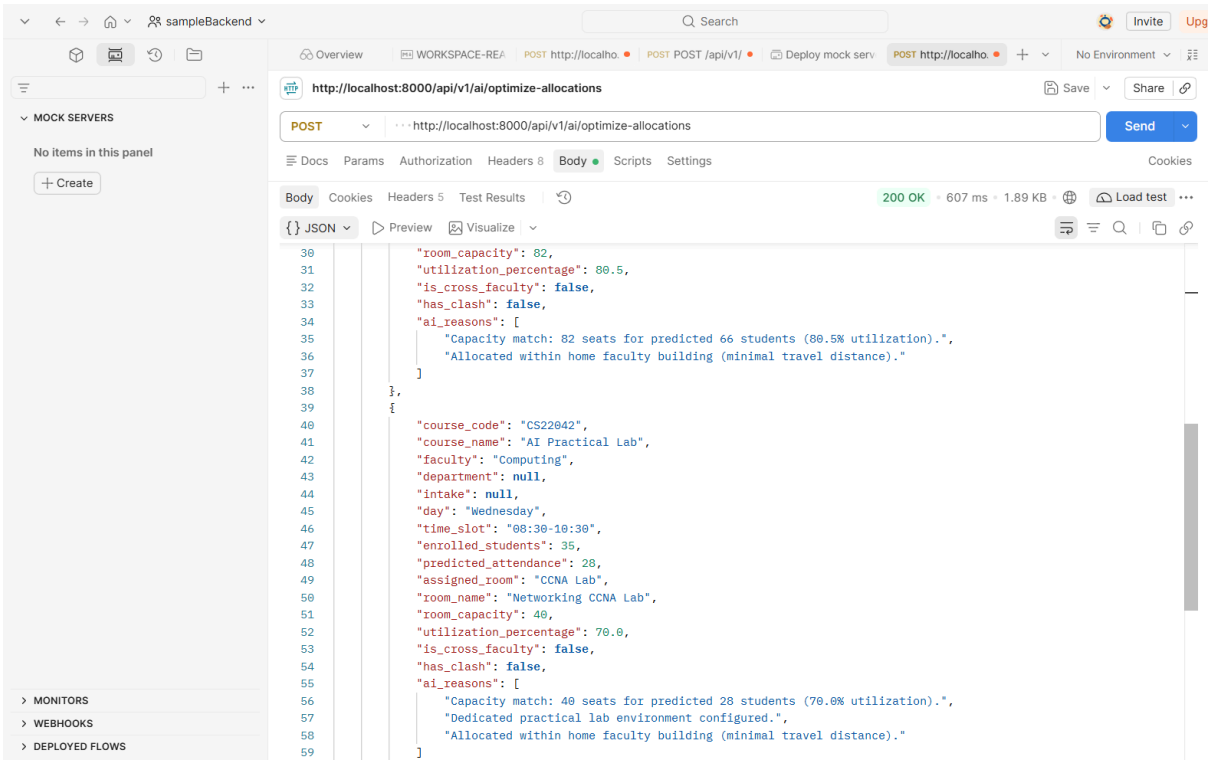


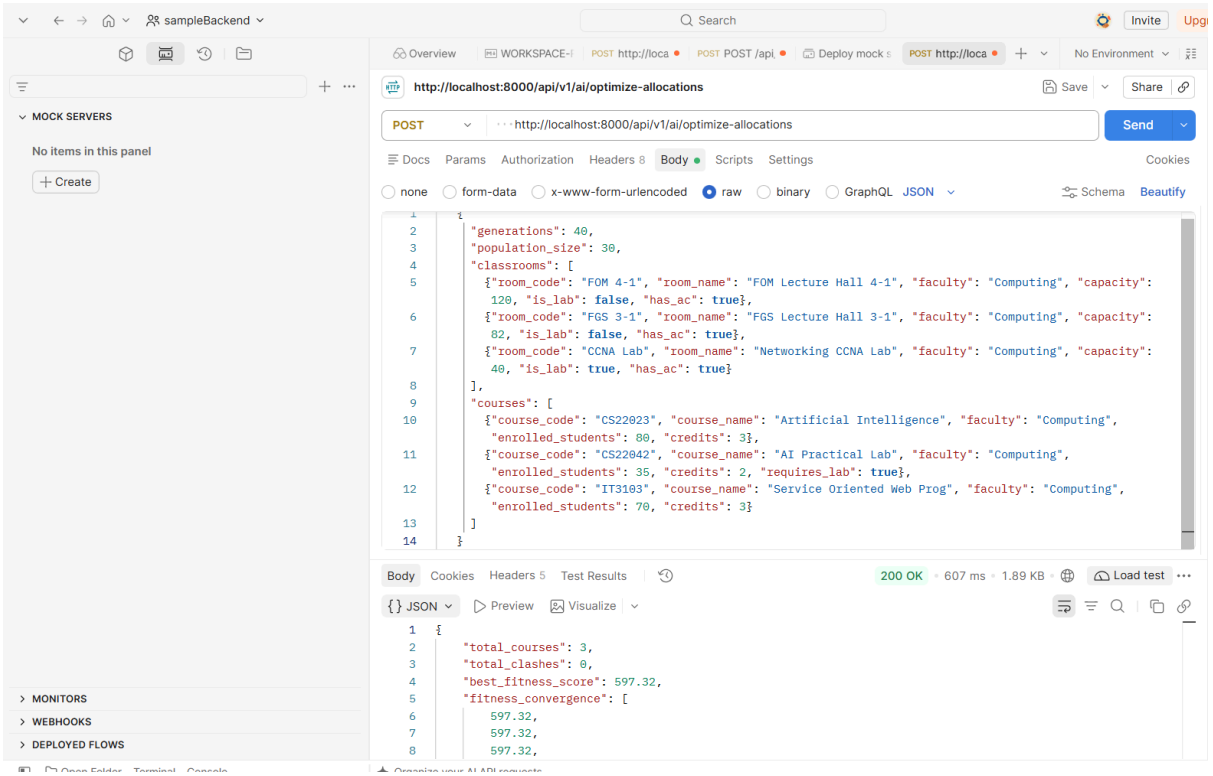
```
Problems  Output  Debug Console  Terminal  Ports

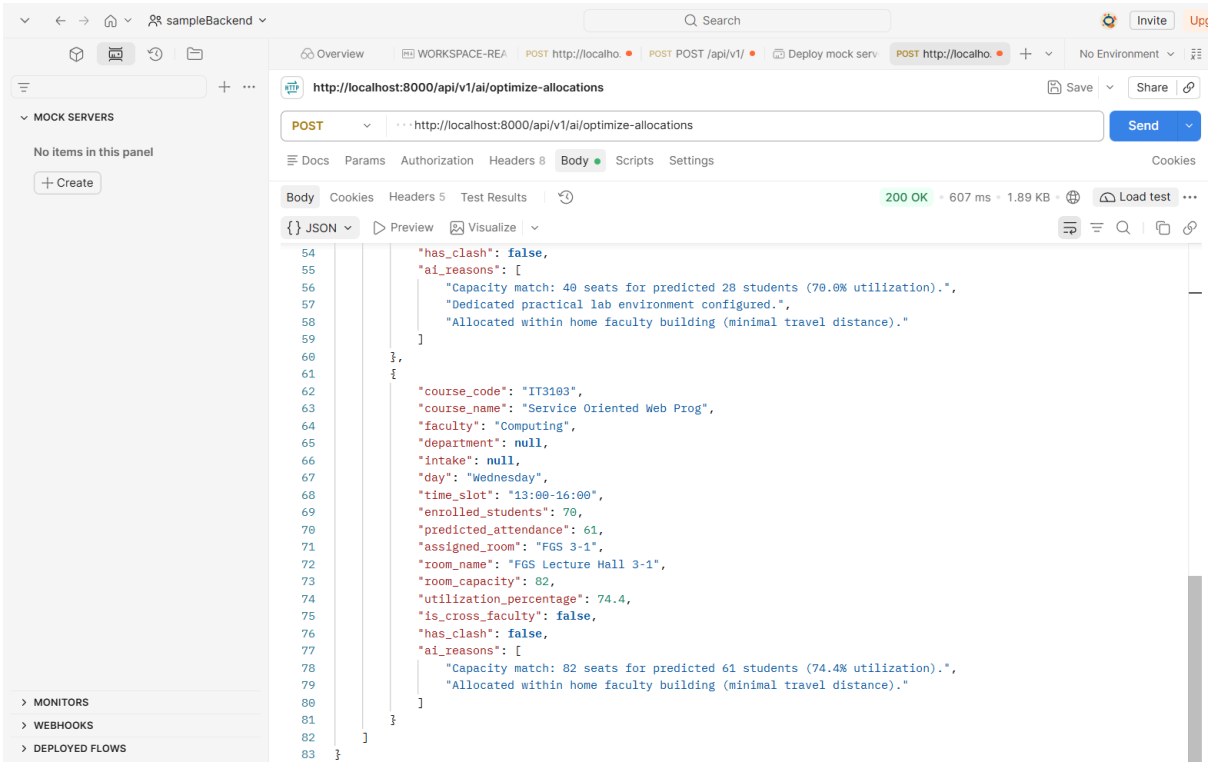
PS C:\Users\Laptopkinglk.com\Downloads\smart-campus-ai> cd C:\Users\Laptopkinglk.com\Downloads\smart-campus-ai\smart-campus-ai\ai-service
>> python -m app.models.genetic_optimizer
Testing Genetic Algorithm Optimizer...
GA Optimization Result Clashes: 0 Fitness: 664.39
PS C:\Users\Laptopkinglk.com\Downloads\smart-campus-ai\smart-campus-ai\ai-service>
```

Shows fitness over generations

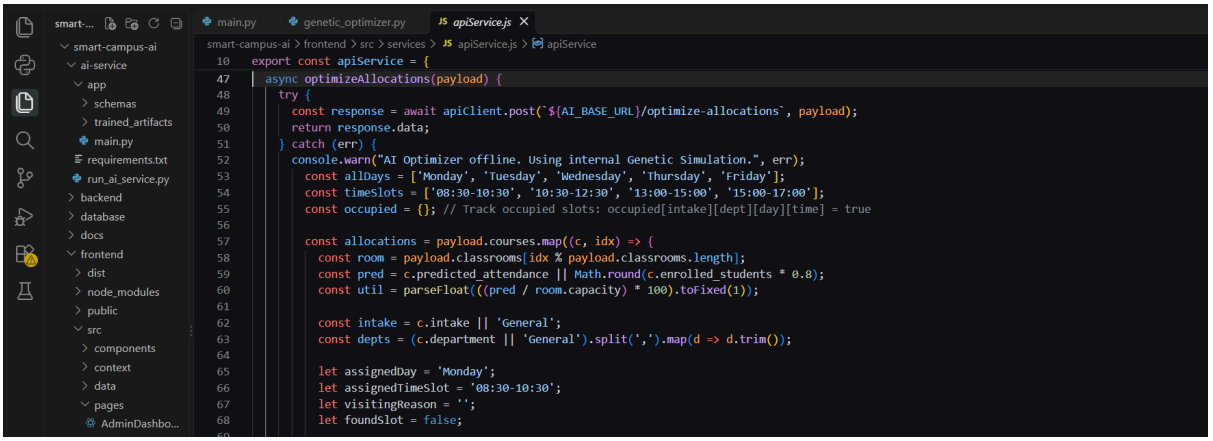


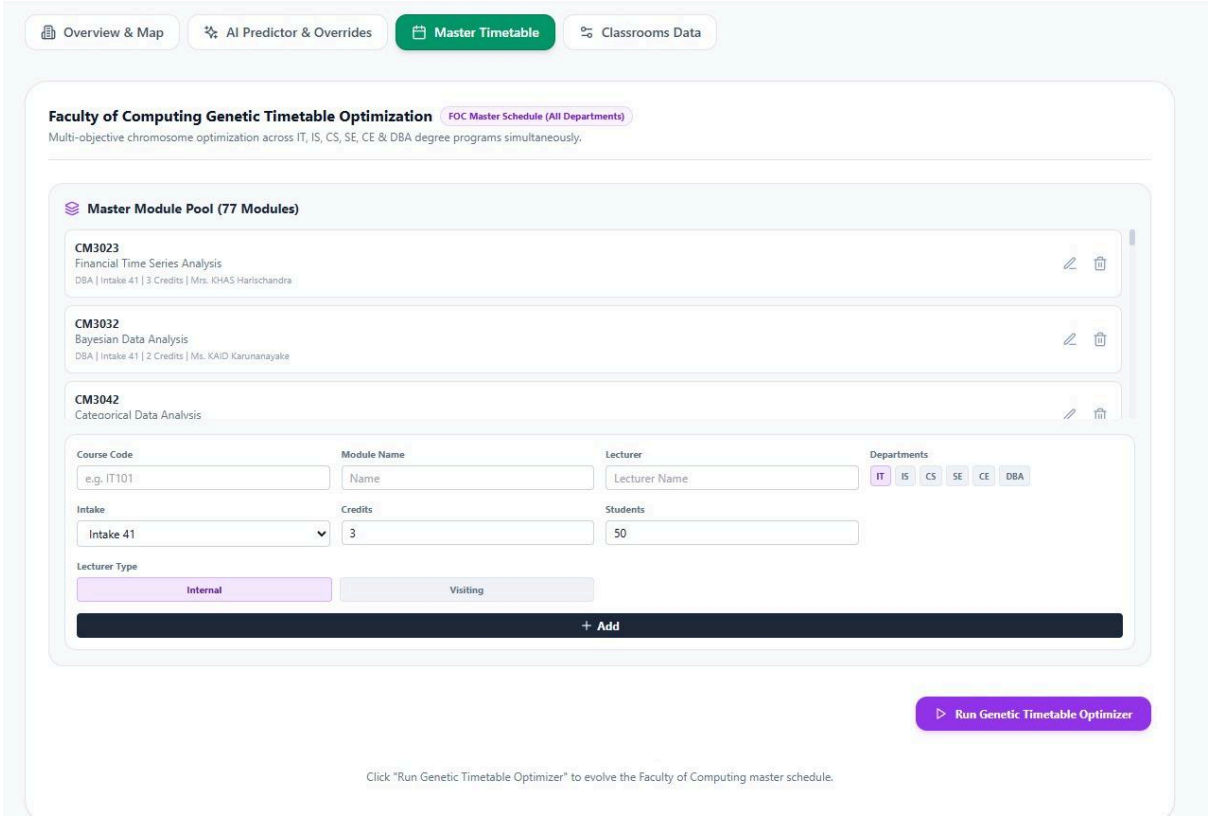



sampleBackend
Search
Invite
Overview
No Environment
MOCK SERVERS
No items in this panel
Create
http://localhost:8000/api/v1/ai/optimize-allocations
Save
Share
POST
http://localhost:8000/api/v1/ai/optimize-allocations
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"generations": 40,
"population_size": 30,
"classrooms": [
{"room_code": "FOM 4-1", "room_name": "FOM Lecture Hall 4-1", "faculty": "Computing", "capacity": 120, "is_lab": false, "has_ac": true},
{"room_code": "FGS 3-1", "room_name": "FGS Lecture Hall 3-1", "faculty": "Computing", "capacity": 82, "is_lab": false, "has_ac": true},
{"room_code": "CCNA Lab", "room_name": "Networking CCNA Lab", "faculty": "Computing", "capacity": 40, "is_lab": true, "has_ac": true}
],
"courses": [
{"course_code": "CS22023", "course_name": "Artificial Intelligence", "faculty": "Computing", "enrolled_students": 80, "credits": 3},
{"course_code": "CS22042", "course_name": "AI Practical Lab", "faculty": "Computing", "enrolled_students": 35, "credits": 2, "requires_lab": true},
{"course_code": "IT3103", "course_name": "Service Oriented Web Prog", "faculty": "Computing", "enrolled_students": 70, "credits": 3}
]
}
Body Cookies Headers 5 Test Results
200 OK 607 ms 1.89 KB
Load test
{ } JSON Preview Visualize
{
"total_courses": 3,
"total_clashes": 0,
"best_fitness_score": 597.32,
"fitness_convergence": [
597.32,
597.32,
597.32,
MONITORS
WEBHOOKS
DEPLOYED FLOWS

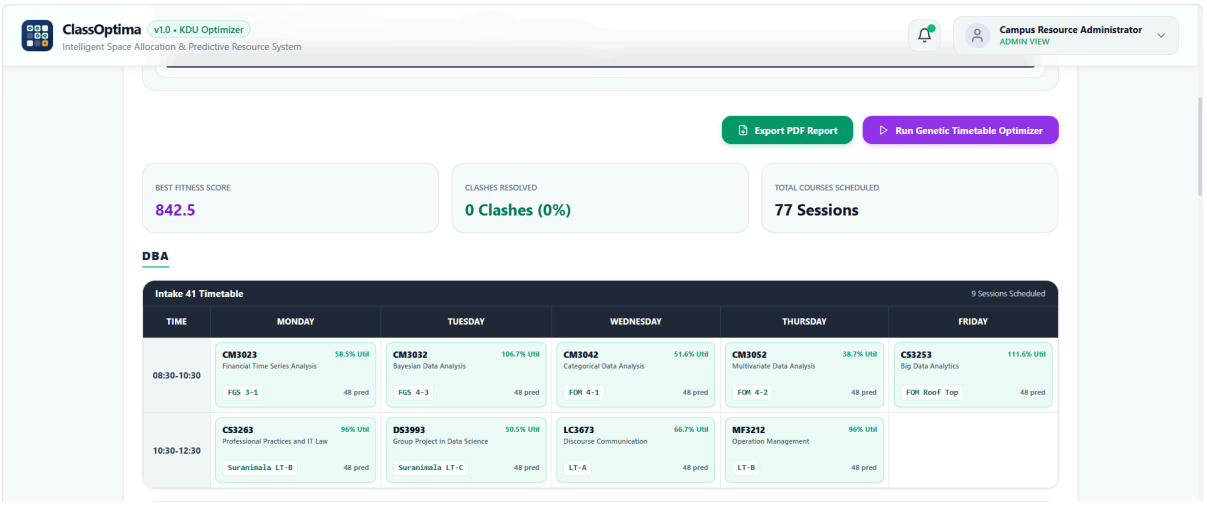

sampleBackend
Search
Invite
Overview
No Environment
MOCK SERVERS
No items in this panel
Create
http://localhost:8000/api/v1/ai/optimize-allocations
Save
Share
POST
http://localhost:8000/api/v1/ai/optimize-allocations
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
Body Cookies Headers 5 Test Results
200 OK 607 ms 1.89 KB
Load test
{ } JSON Preview Visualize
"has_clash": false,
"ai_reasons": [
"Capacity match: 40 seats for predicted 28 students (70.0% utilization).",
"Dedicated practical lab environment configured.",
"Allocated within home faculty building (minimal travel distance)."
]
},
{
"course_code": "IT3103",
"course_name": "Service Oriented Web Prog",
"faculty": "Computing",
"department": null,
"intake": null,
"day": "Wednesday",
"time_slot": "13:00-16:00",
"enrolled_students": 70,
"predicted_attendance": 61,
"assigned_room": "FGS 3-1",
"room_name": "FGS Lecture Hall 3-1",
"room_capacity": 82,
"utilization_percentage": 74.4,
"is_cross_faculty": false,
"has_clash": false,
"ai_reasons": [
"Capacity match: 82 seats for predicted 61 students (74.4% utilization).",
"Allocated within home faculty building (minimal travel distance)."
]
}
]
}
MONITORS
WEBHOOKS
DEPLOYED FLOWS

Fallback when the AI service is unavailable






842.5
0 Clashes (0%)
77 Sessions



Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student : HDMC Nawarathna **Date:** 17/10/2026

Signature:



Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**HMHP Herath**	**Registration No.**	**D/BIT/24/0069**
Degree / Intake	**Information Technology** **41**	**Group No.**	**31**
Project Title	**AI-Powered University Classroom Utilization Optimizer**		

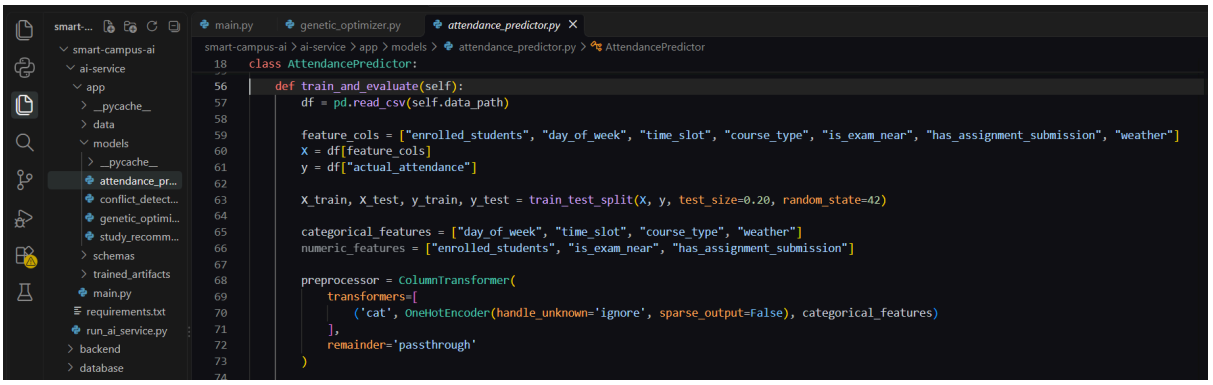
Technical Contributions:

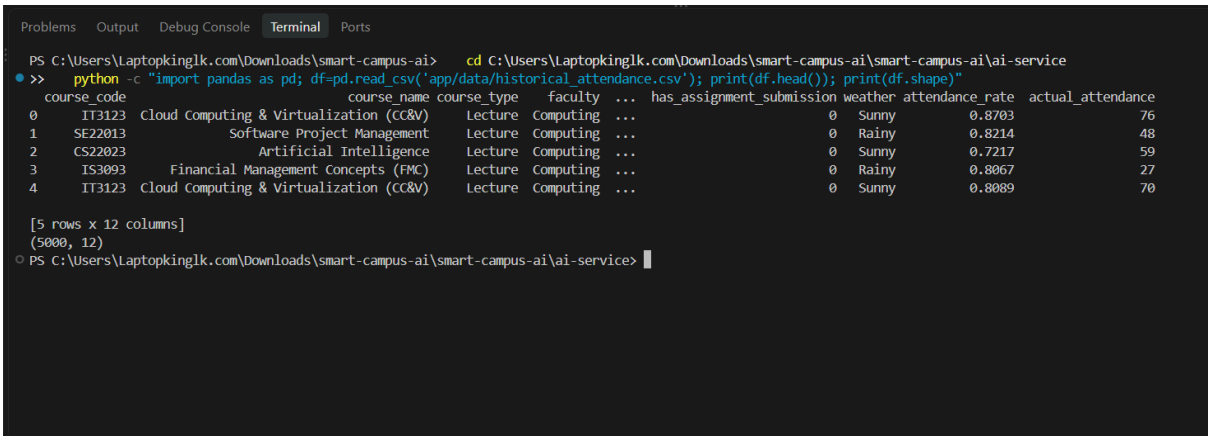
Task / Component	My Contribution	Tools / AI Technique Used
Student Attendance Prediction.	**Supervised Machine Learning –Regression** Developed and evaluated a Random Forest Regressor to predict student attendance using historical academic session data.	● Python ● Random Forest ● Regressor ● Regression
Data Processing and Feature Engineering.	**Data Preprocessing and Feature Engineering** Prepared academic session records, performed data cleaning and feature preparation,selected seven input features.	● Pandas ● NumPy ● Data cleaning ● Feature Engineering
Explainable artificial Intelligent	**Model Interpretability – Feature Importance** Extracted and interpreted feature importance information from the trained model to help administrators understand the factors Influencing attendance predictions.	● Explainable AI ● RAndom Forest Feature importance ● Python

Constraint based rule search and intelligent decision making	**Multi-Criteria Decision Making (MCDM) – MAUT** Contributed to evaluating alternatives using multiple criteria for study space recommendations and emergency classroom allocation, Considering factors such as availability,capacity, AC availability and suitability	• Multi Attribute utility theory, • Multi criteria decision making, • Constraint Based Rules
AI Model Integration and system Support	**Machine Learning Model Integration** Prepared attendance prediction outputs for use by the classroom optimization component, Supporting classroom allocation based on predicted attendance rather than enrollment figure alone.	• Python • AI service Integration • Attendance prediction

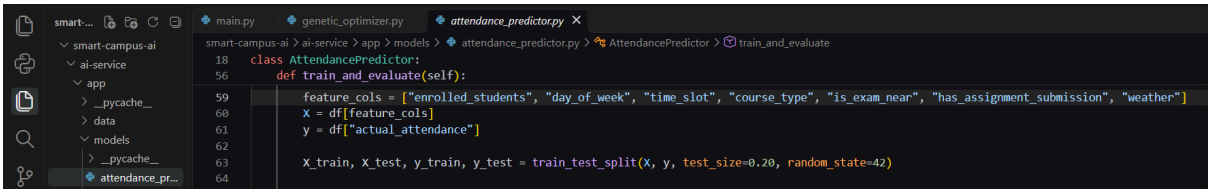
Evidence/Output

Dataset loading and preparation

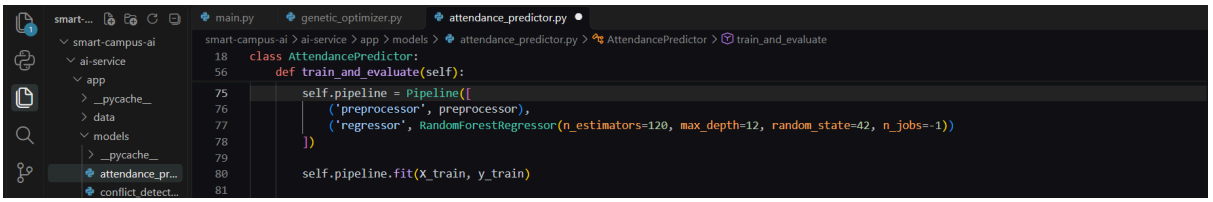




Feature selection and train-test split

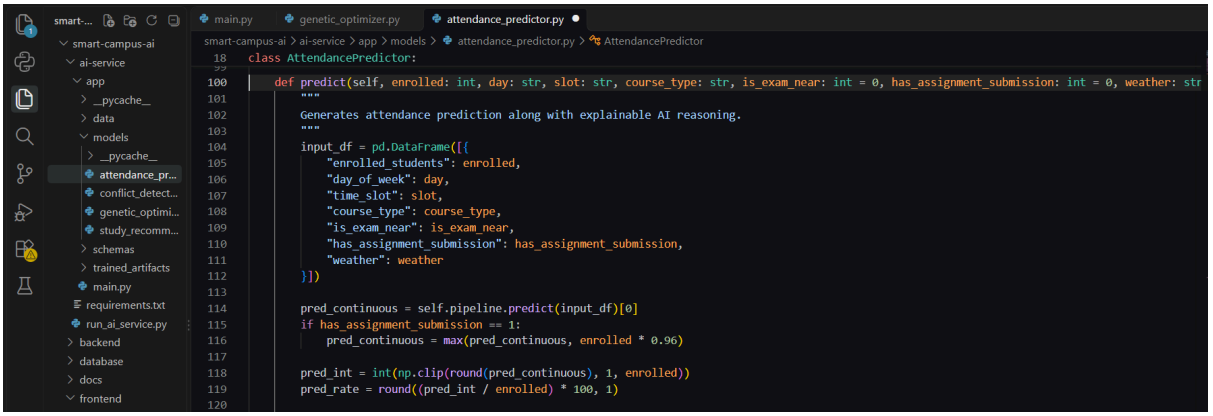


Random Forest model

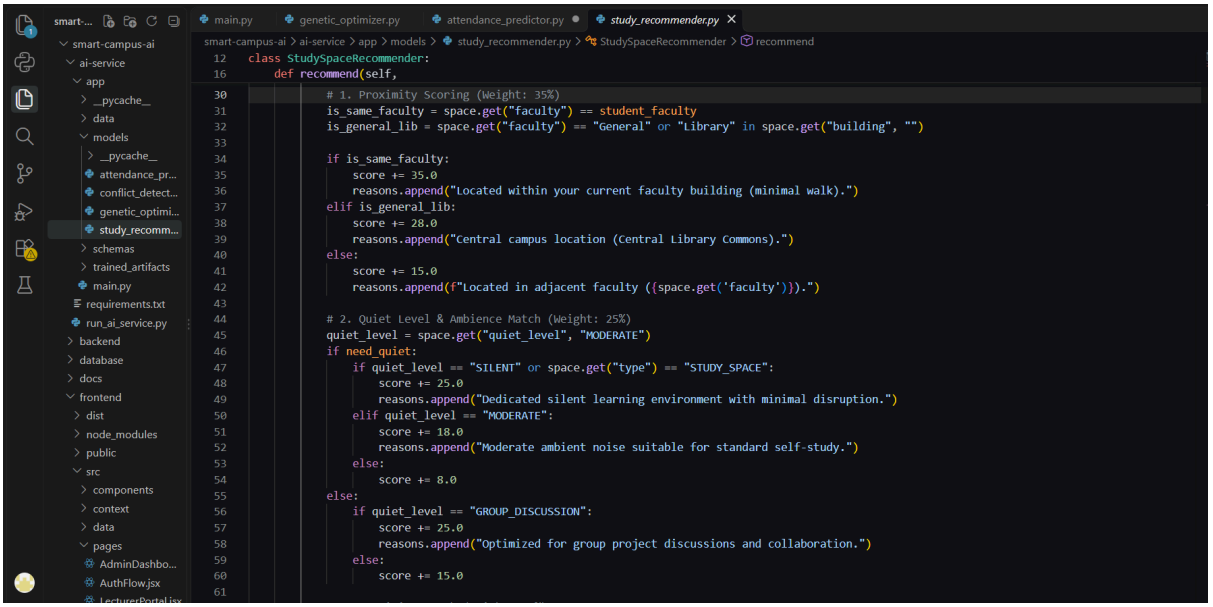


ML pipeline implementation





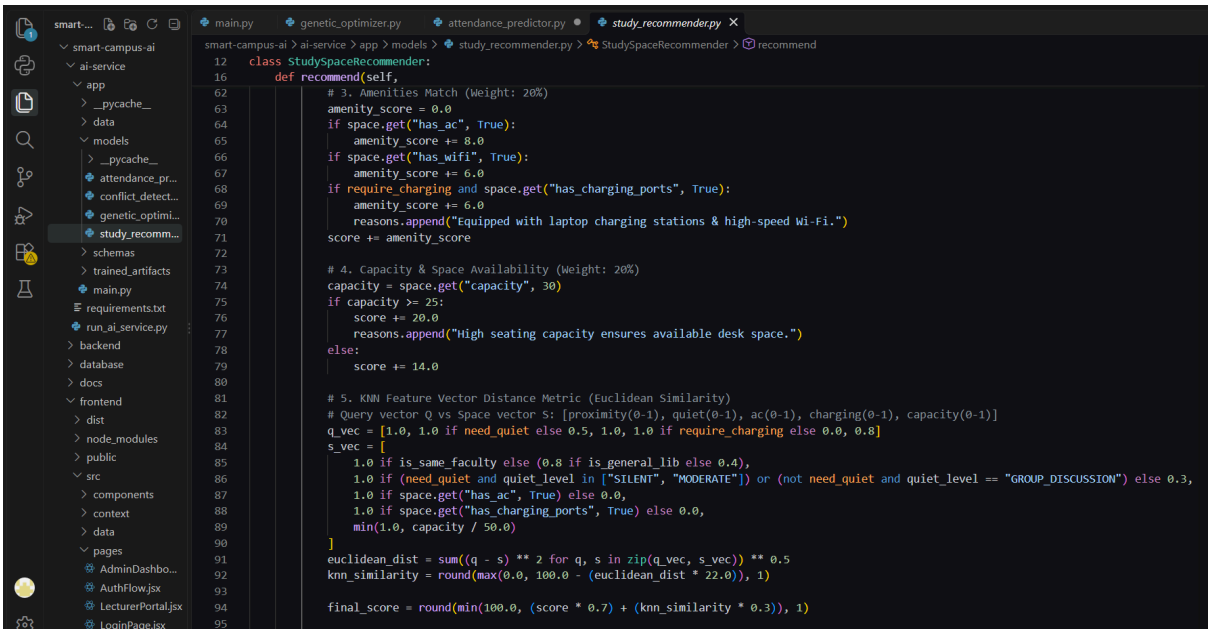
MAUT/MCDM weights and utility calculation



```python
class StudySpaceRecommender:
    def recommend(self,
        # 1. Proximity Scoring (Weight: 35%)
        is_same_faculty = space.get("faculty") == student_faculty
        is_general_lib = space.get("faculty") == "General" or "Library" in space.get("building", "")

        if is_same_faculty:
            score += 35.0
            reasons.append("Located within your current faculty building (minimal walk).")
        elif is_general_lib:
            score += 28.0
            reasons.append("Central campus location (Central Library Commons).")
        else:
            score += 15.0
            reasons.append(f"Located in adjacent faculty ({space.get('faculty')}).")

        # 2. Quiet Level & Ambience Match (Weight: 25%)
        quiet_level = space.get("quiet_level", "MODERATE")
        if need_quiet:
            if quiet_level == "SILENT" or space.get("type") == "STUDY_SPACE":
                score += 25.0
                reasons.append("Dedicated silent learning environment with minimal disruption.")
            elif quiet_level == "MODERATE":
                score += 18.0
                reasons.append("Moderate ambient noise suitable for standard self-study.")
            else:
                score += 8.0
        else:
            if quiet_level == "GROUP_DISCUSSION":
                score += 25.0
                reasons.append("Optimized for group project discussions and collaboration.")
            else:
                score += 15.0
```



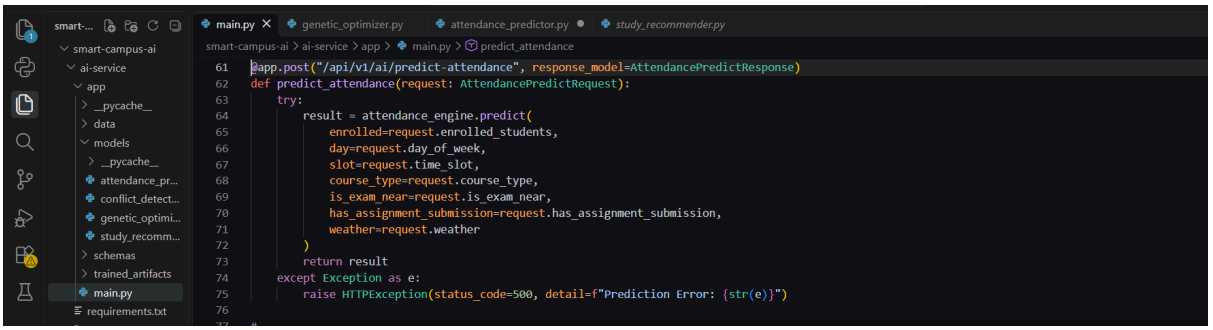
```python
class StudySpaceRecommender:
    def recommend(self,
        # 3. Amenities Match (Weight: 20%)
        amenity_score = 0.0
        if space.get("has_ac", True):
            amenity_score += 8.0
        if space.get("has_wifi", True):
            amenity_score += 6.0
        if require_charging and space.get("has_charging_ports", True):
            amenity_score += 6.0
            reasons.append("Equipped with laptop charging stations & high-speed Wi-Fi.")
        score += amenity_score

        # 4. Capacity & Space Availability (Weight: 20%)
        capacity = space.get("capacity", 30)
        if capacity >= 25:
            score += 20.0
            reasons.append("High seating capacity ensures available desk space.")
        else:
            score += 14.0

        # 5. KNN Feature Vector Distance Metric (Euclidean Similarity)
        # Query vector Q vs Space vector S: [proximity(0-1), quiet(0-1), ac(0-1), charging(0-1), capacity(0-1)]
        q_vec = [1.0, 1.0 if need_quiet else 0.5, 1.0, 1.0 if require_charging else 0.0, 0.8]
        s_vec = [
            1.0 if is_same_faculty else (0.8 if is_general_lib else 0.4),
            1.0 if (need_quiet and quiet_level in ["SILENT", "MODERATE"]) or (not need_quiet and quiet_level == "GROUP_DISCUSSION") else 0.3,
            1.0 if space.get("has_ac", True) else 0.0,
            1.0 if space.get("has_charging_ports", True) else 0.0,
            min(1.0, capacity / 50.0)
        ]
        euclidean_dist = sum((q - s) ** 2 for q, s in zip(q_vec, s_vec)) ** 0.5
        knn_similarity = round(max(0.0, 100.0 - (euclidean_dist * 22.0)), 1)

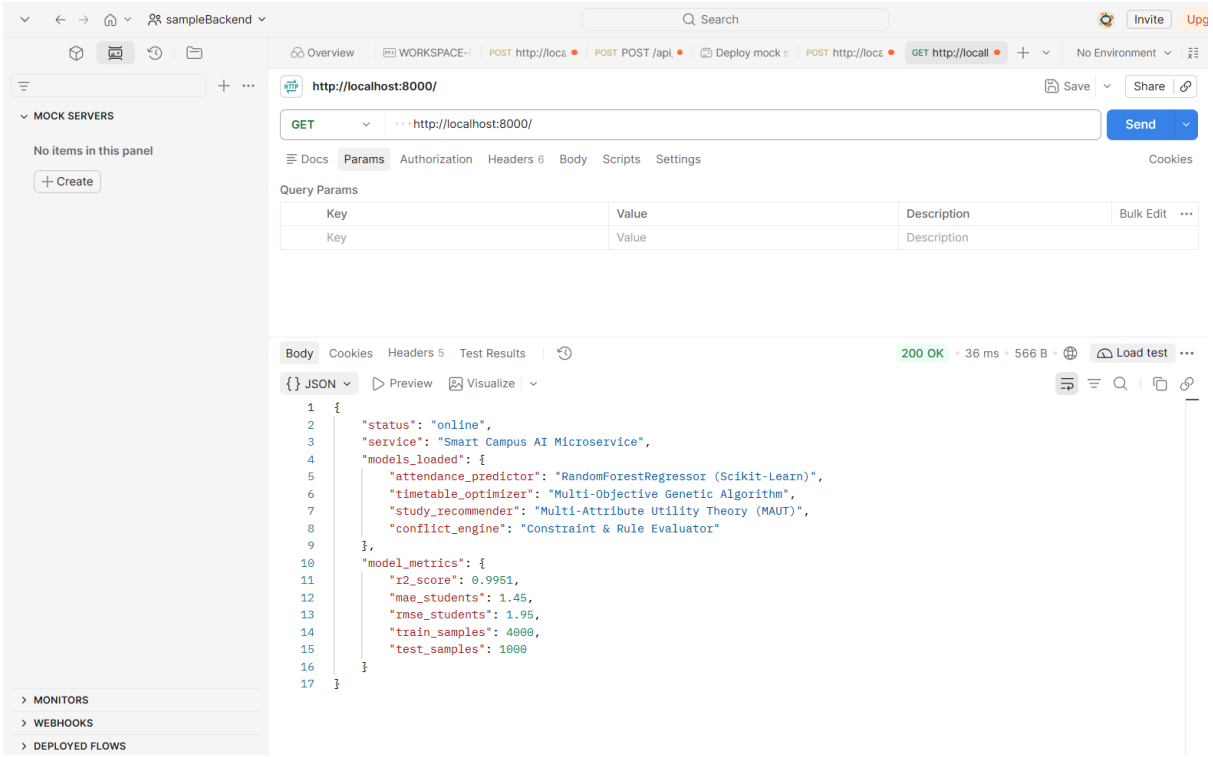
        final_score = round(min(100.0, (score * 0.7) + (knn_similarity * 0.3)), 1)
```

Prediction API integration

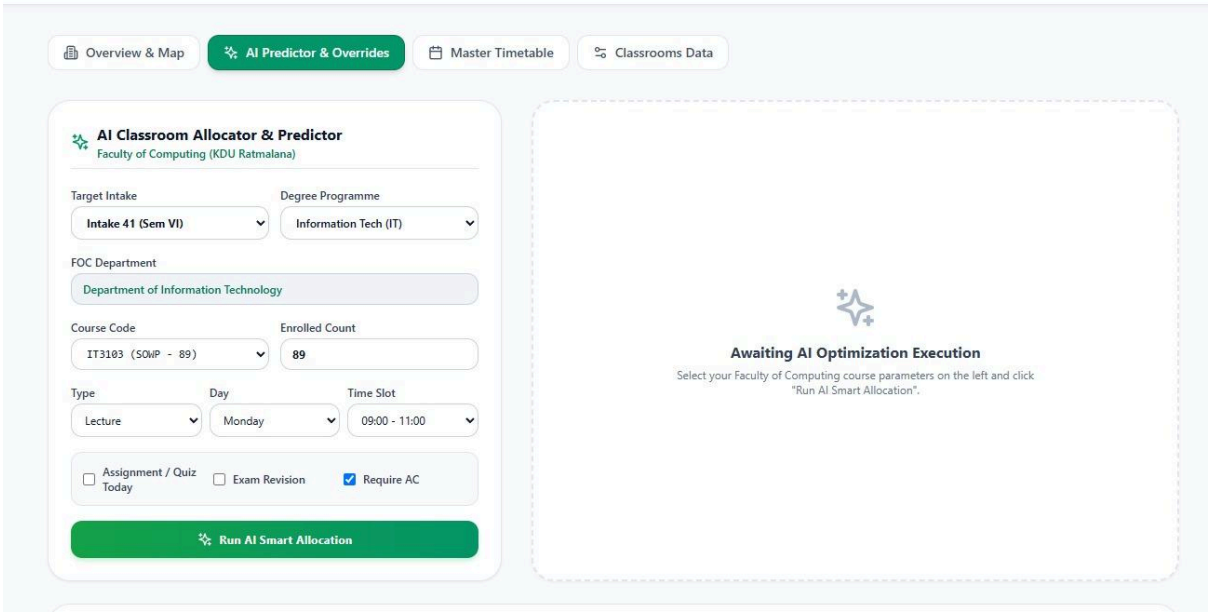


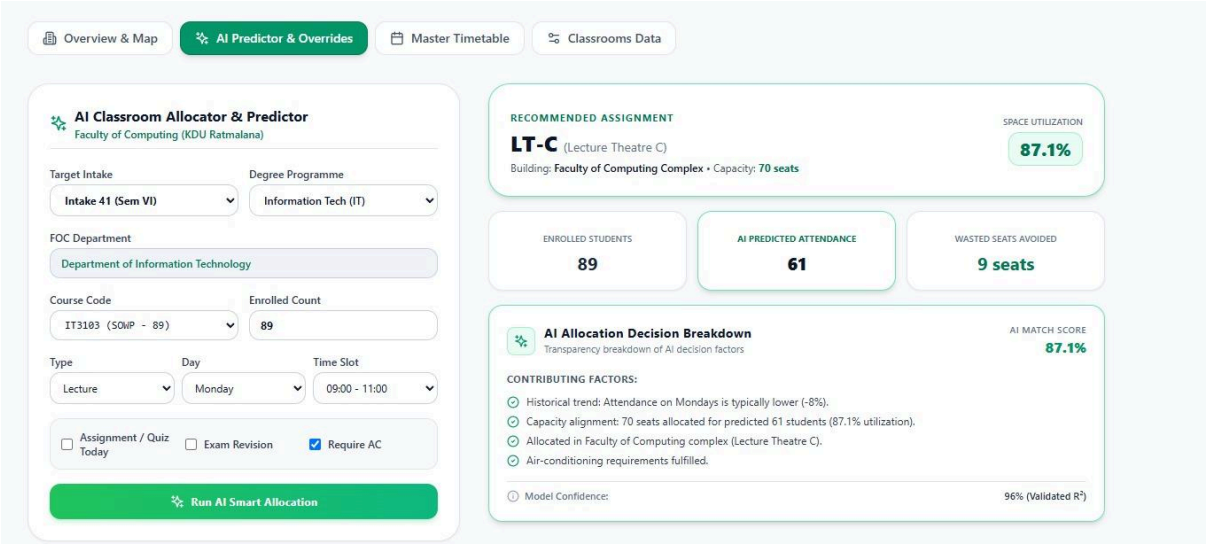
```python
@app.post("/api/v1/ai/predict-attendance", response_model=AttendancePredictResponse)
def predict_attendance(request: AttendancePredictRequest):
    try:
        result = attendance_engine.predict(
            enrolled=request.enrolled_students,
            day=request.day_of_week,
            slot=request.time_slot,
            course_type=request.course_type,
            is_exam_near=request.is_exam_near,
            has_assignment_submission=request.has_assignment_submission,
            weather=request.weather
        )
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Prediction Error: {str(e)}")
```

Evaluation results (R^2 and MAE)



Integrated prediction and classroom recommendation





Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student : HMHP Herath **Date:** 17/10/2026

Signature:



Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**LSR Gunathunga**	**Registration No.**	**D/BCS/25/0015**
Degree / Intake	**CS/42**	**Group No.**	**31**
Project Title	**AI-Powered University Classroom Utilization Optimizer**		

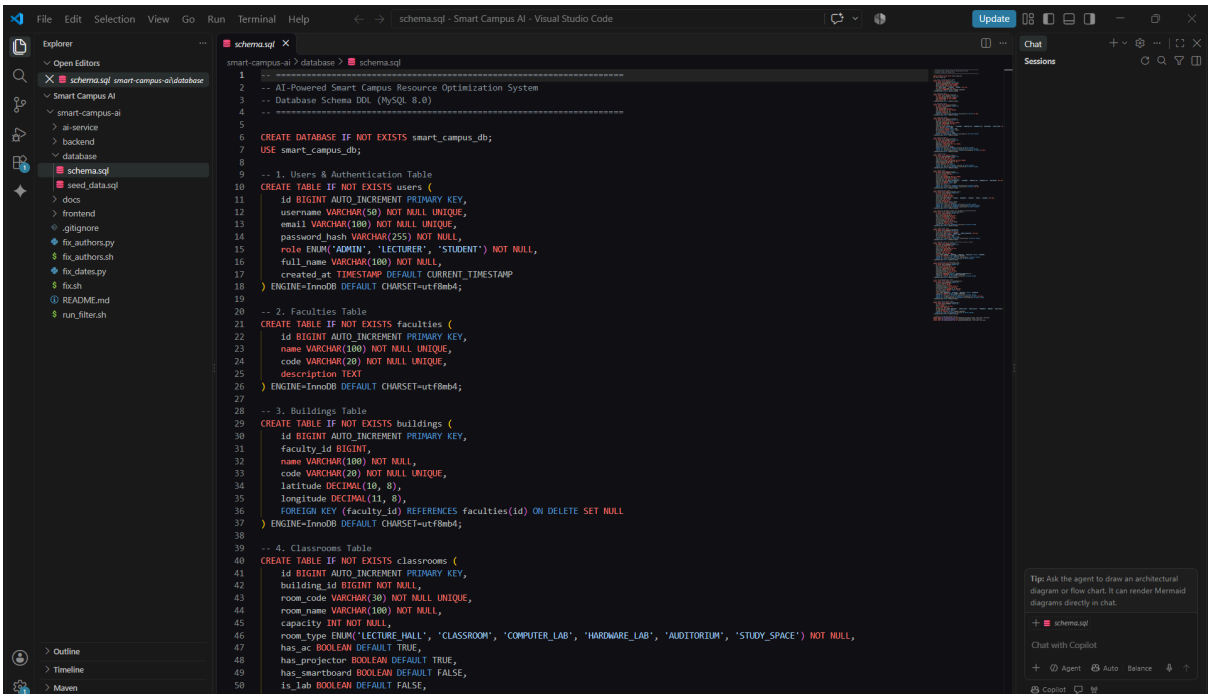
Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
Database Architecture and Design	Designed the relational database structure using MySQL 8.0, identifying and defining core entities (faculties, departments, courses, classrooms, capacities, facilities, and timetable slots) to maintain data consistency.	MySQL 8.0, SQL, Relational Database Design	Relational database schema script (schema.sql)
Spring Boot Backend & ORM Integration	Developed backend entity classes and repositories using Spring Boot and Hibernate/JPA.	Java, Spring Boot, Hibernate, JPA	Backend entity classes and repository packages (backend/src/main/java/com/campus/optimizer)
REST API Development	Developed and managed RESTful API endpoints within the Spring Boot backend to provide efficient data access for classrooms, courses,	Spring Boot, REST APIs, JSON	Controller classes (TimetableController.java, ClassroomController.java)

	and schedules.		
Lecturer Schedule Management Integration	Implemented backend logic and support to serve personalized, dynamic timetable views and schedule updates specifically for academic staff interacting with the system.	Spring Boot, REST APIs	Backend integration code and lecturer portal endpoints

Evidence/ Output

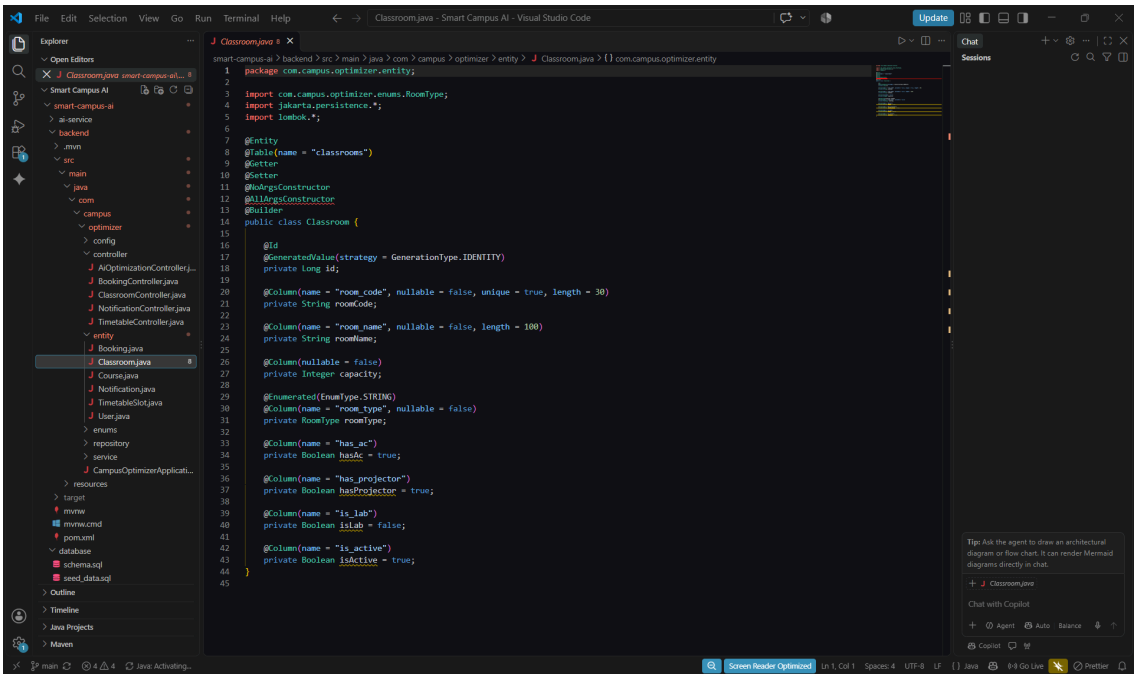
1. **Database Architecture and Design**

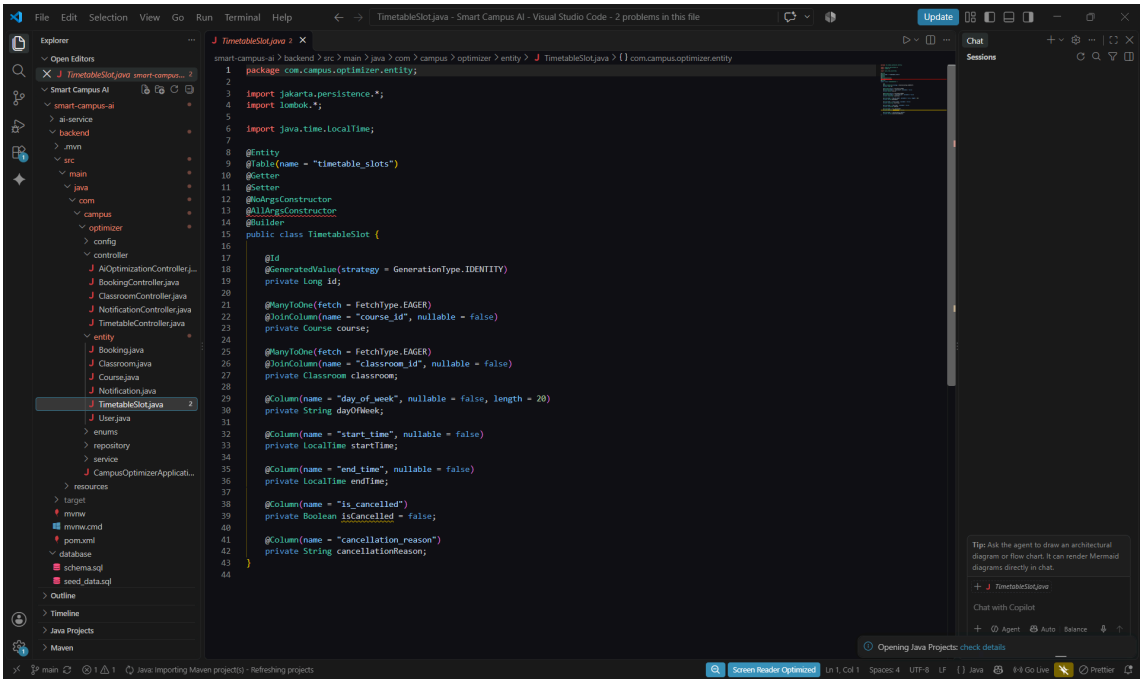


VS Code source code view of schema.sql demonstrating the relational database architecture designed for the AI-Powered University Classroom Utilization Optimizer, showing core entities (users, faculties, buildings, and classrooms) with proper primary keys, foreign keys, and ENUM constraints.

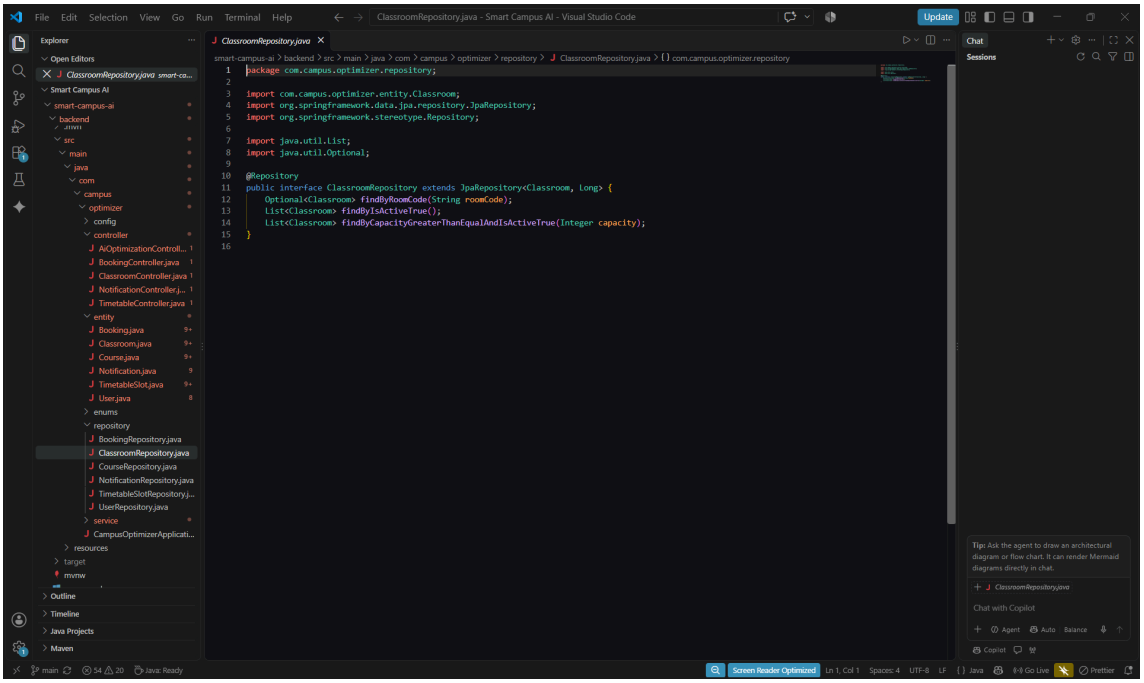
2. Spring Boot Backend & ORM Integration

Entity Mapping Evidence

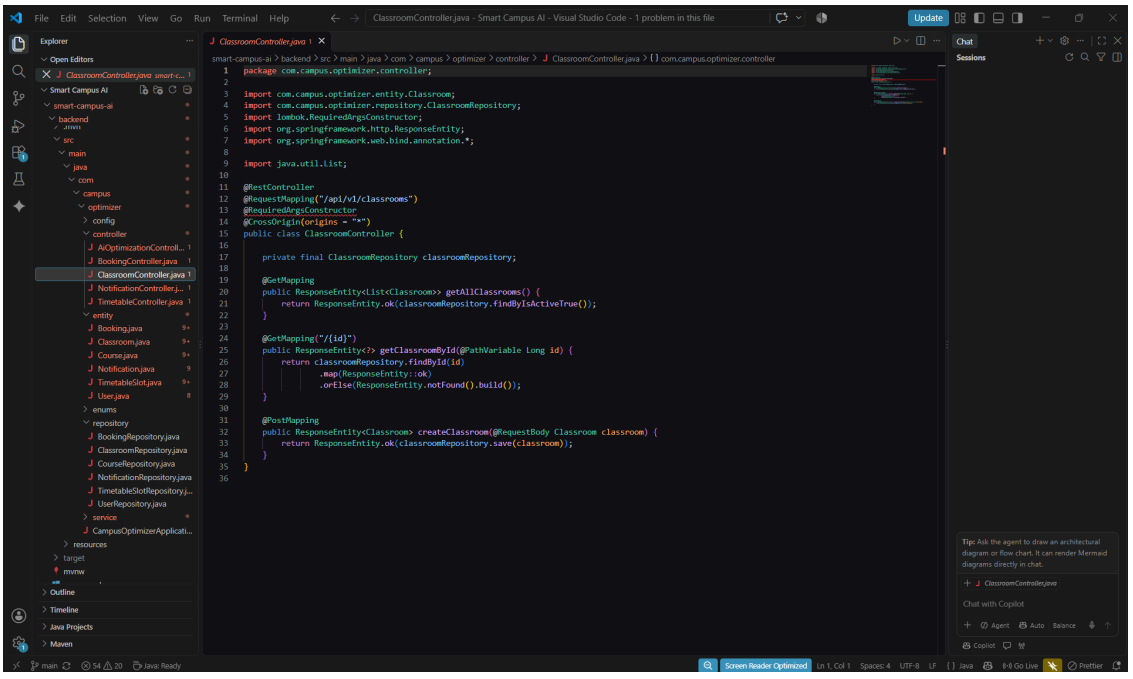


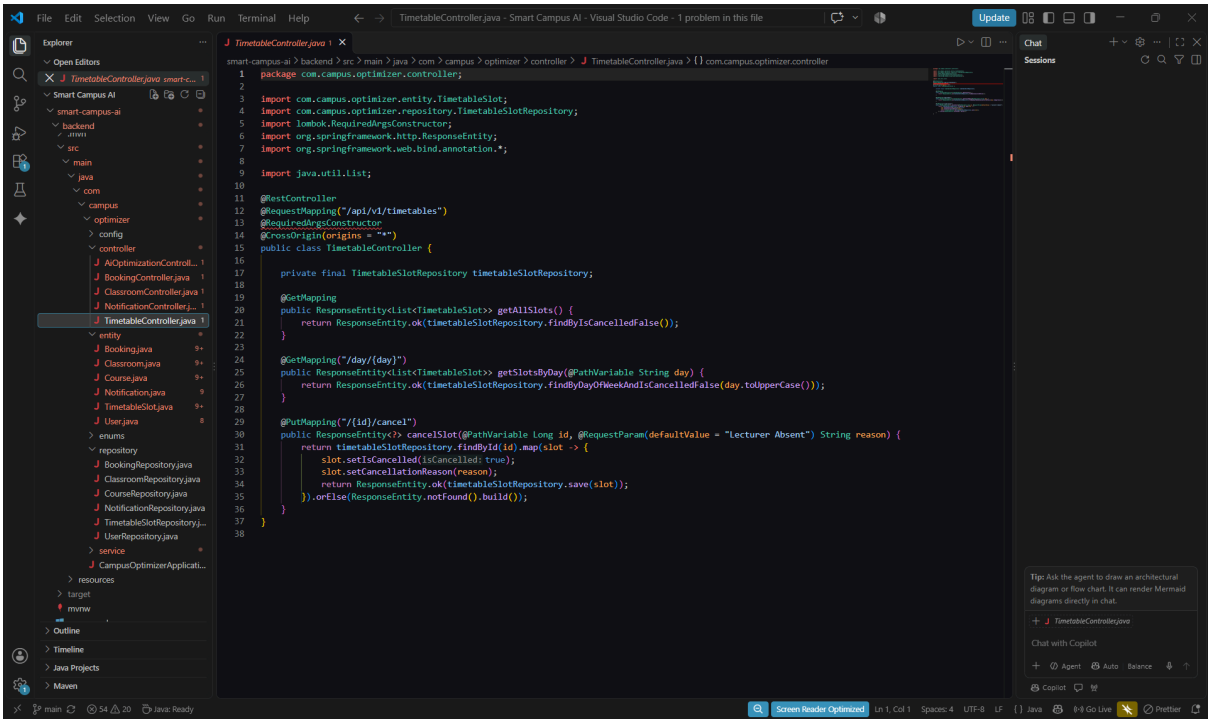


).



3. REST API Development






```java
package com.campus.optimizer.controller;

import com.campus.optimizer.entity.TimetableSlot;
import com.campus.optimizer.repository.TimetableSlotRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/timetables")
@RequiredArgsConstructor
@CrossOrigin(origins = "*")
public class TimetableController {

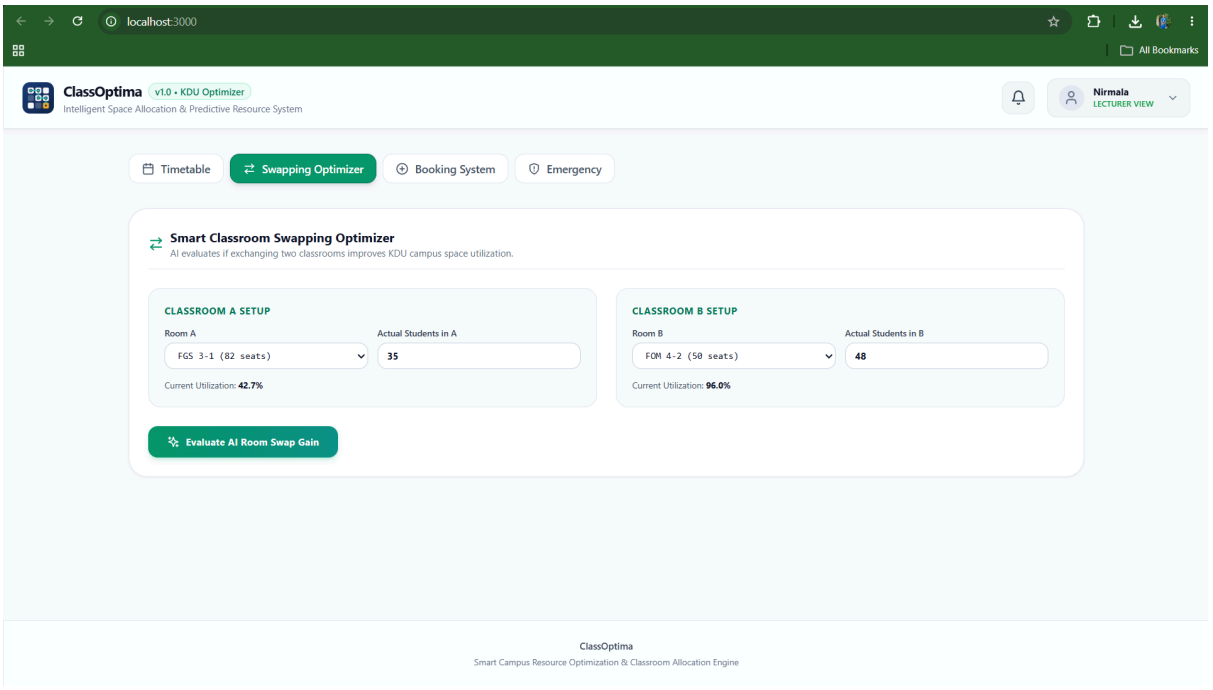
    private final TimetableSlotRepository timetableSlotRepository;

    @GetMapping
    public ResponseEntity<List<TimetableSlot>> getAllSlots() {
        return ResponseEntity.ok(timetableSlotRepository.findByIsCancelledFalse());
    }

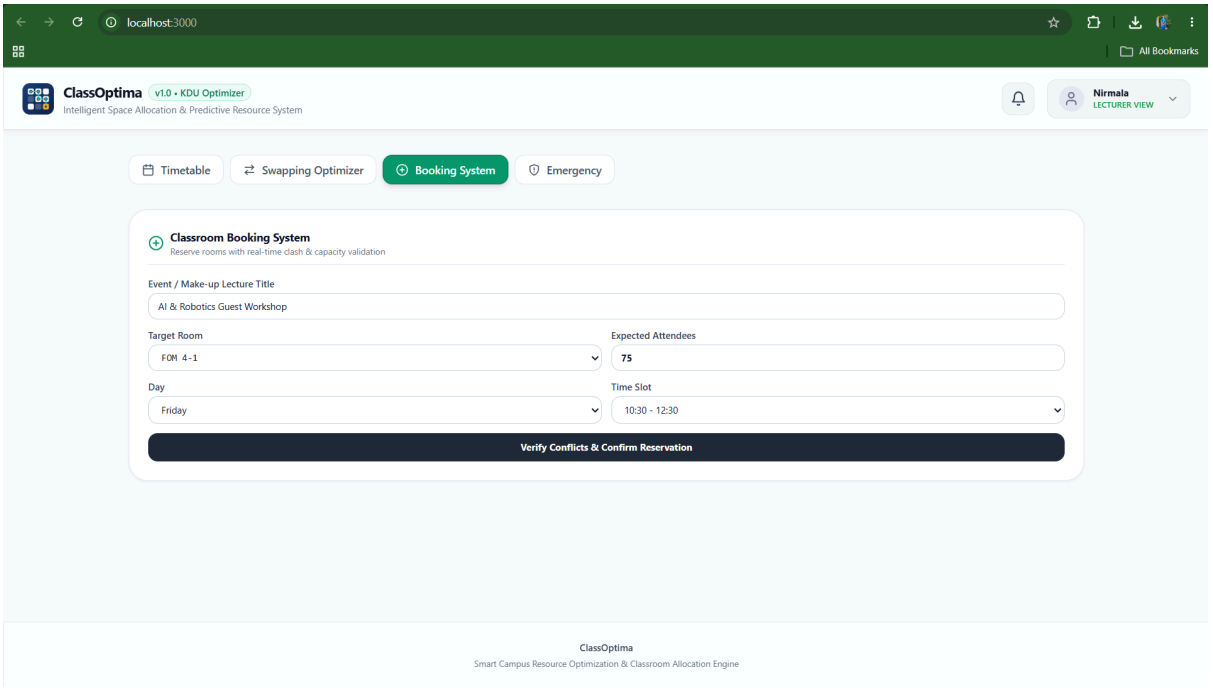
    @GetMapping("/day/{day}")
    public ResponseEntity<List<TimetableSlot>> getSlotsByDay(@PathVariable String day) {
        return ResponseEntity.ok(timetableSlotRepository.findByDayOfWeekAndIsCancelledFalse(day.toUpperCase()));
    }

    @PutMapping("/{id}/cancel")
    public ResponseEntity<?> cancelSlot(@PathVariable Long id, @RequestParam(defaultValue = "Lecturer Absent") String reason) {
        return timetableSlotRepository.findById(id).map(slot -> {
            slot.setIsCancelled(isCancelled: true);
            slot.setCancellationReason(reason);
            return ResponseEntity.ok(timetableSlotRepository.save(slot));
        }).orElse(ResponseEntity.notFound().build());
    }
}
```

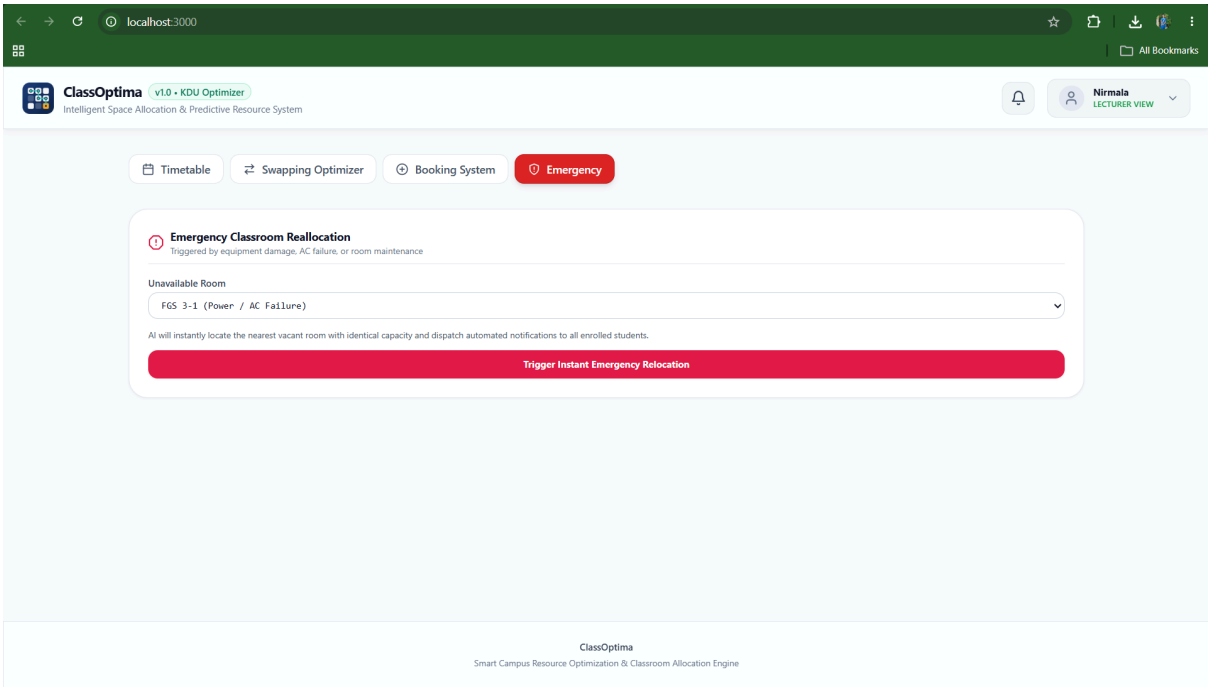
4. Lecturer Schedule Management Integration



Lecturer Portal UI demonstrating the Smart Classroom Swapping Optimizer interface, showing role-based authentication (Lecturer View) and classroom capacity/utilization analysis parameters managed through the system.



Lecturer Portal UI showing the Classroom Booking System interface, which allows instructors to reserve rooms with real-time clash and capacity validation for events and make-up lectures.



Lecturer Portal UI highlighting the Emergency Classroom Reallocation module, designed to handle unexpected room failures (e.g., power or AC failure) and instantly locate vacant rooms with automated notifications

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student	LSR Gunathunga	**Date:**	17/10/2026

Signature:



Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.